

Deep Learning for Computer Vision

Lecture 9: Vision Transformers

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen



cppape

<https://user.informatik.uni-goettingen.de/~pape41/>

Deep Learning for Computer Vision

Lecture 9: Vision Transformers

Constantin Pape

Institute of Computer Science, Georg August Universität Göttingen



@cppape.bsky.social

<https://user.informatik.uni-goettingen.de/~pape41/>

Agenda for today

- Sequence to Sequence Models, Self-Attention & Transformers
- Transformers in NLP
 - Large Language Models
- Vision Transformers

Exercises & Sample Exam

- Exercise 4 (U-Net): Was due today. Exercises are this week.
- Exercise 5 (Variational Auto Encoder):
 - Exercise will be published today.
 - Due on the 20th of January.
- Exercise 6: Will be published on the 20th of January, due on February 3rd.
- Sample Exam: We will make it available around 20th of January.
 - Will be discussed in the last lecture.

Sequence to Sequence Models, Self-Attention & Transformers

Sequence to Sequence Models

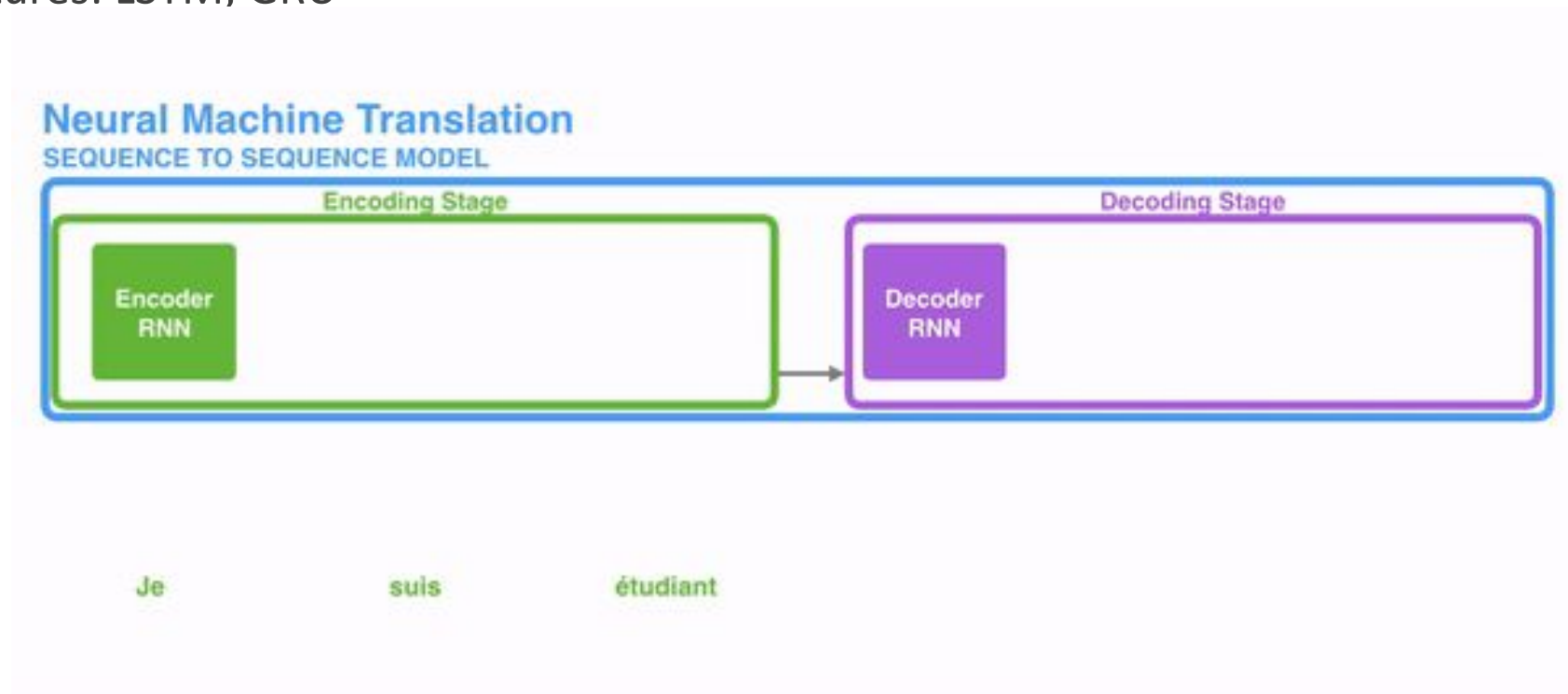
Sequence to sequence models: transform input sequence to output sequence

- Fundamental task in NLP
 - Translation
 - Text generation
 - ...
- Increasingly popular in Computer Vision (due to transformer architecture)
 - second part of this lecture

Recurrent Neural Networks (RNN)

RNN:

- Process sequence as time series
- Keep *hidden state* that encodes past
- Architectures: LSTM, GRU



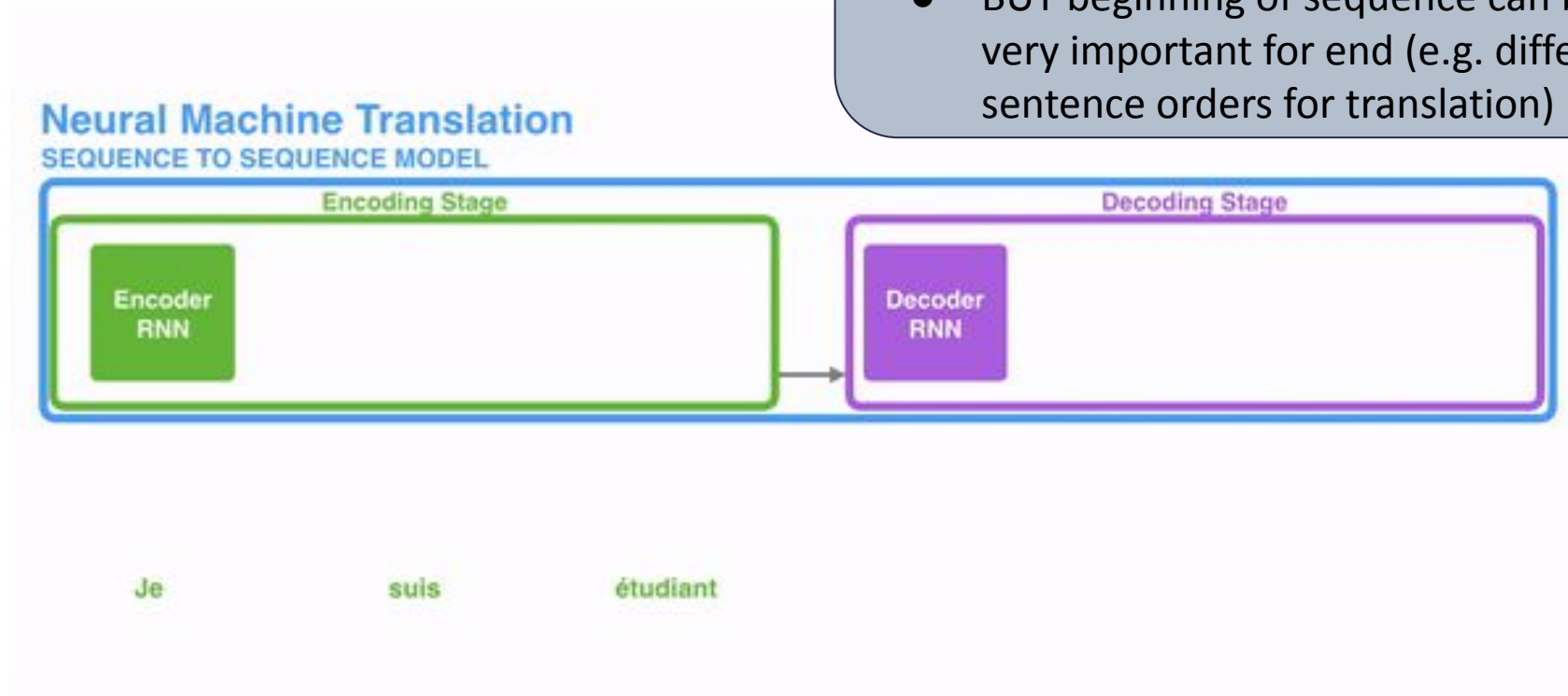
Recurrent Neural Networks (RNN)

RNN:

- Process sequence as time series
- Keep *hidden state* that encodes past

Problem:

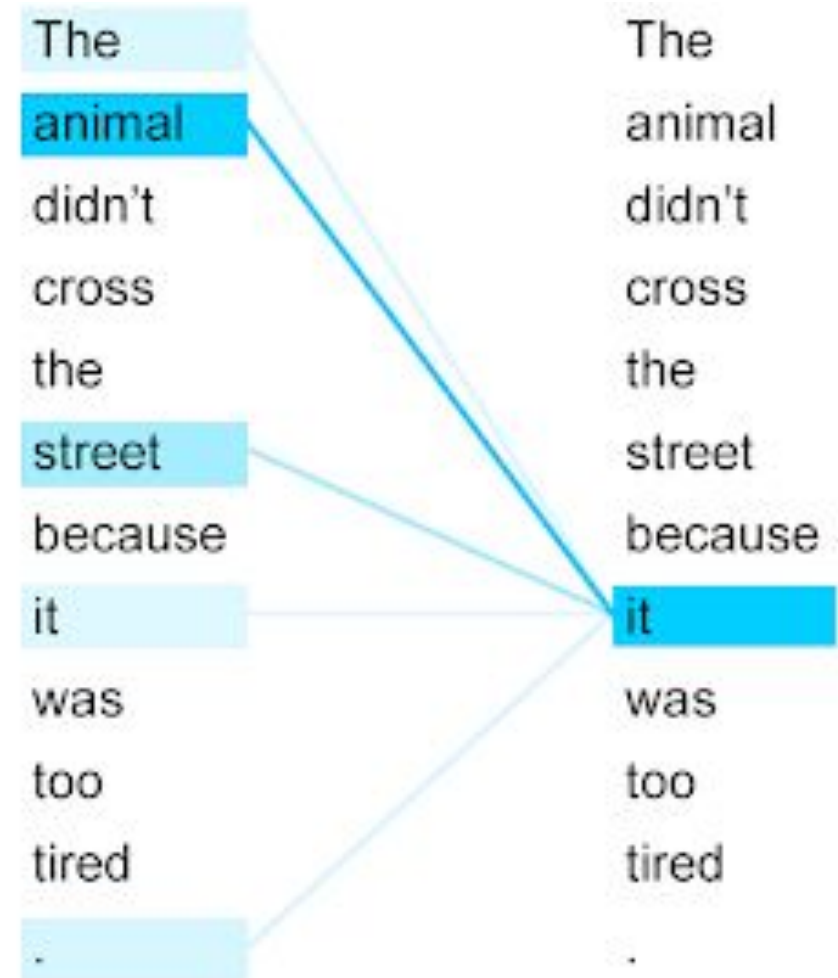
- influence of past sequence elements decreases over sequence
- BUT beginning of sequence can be very important for end (e.g. different sentence orders for translation)



Self-Attention

Self-attention:

- every element in sequence can influence every other element
- learn weighting (“attention”) for each pair of elements



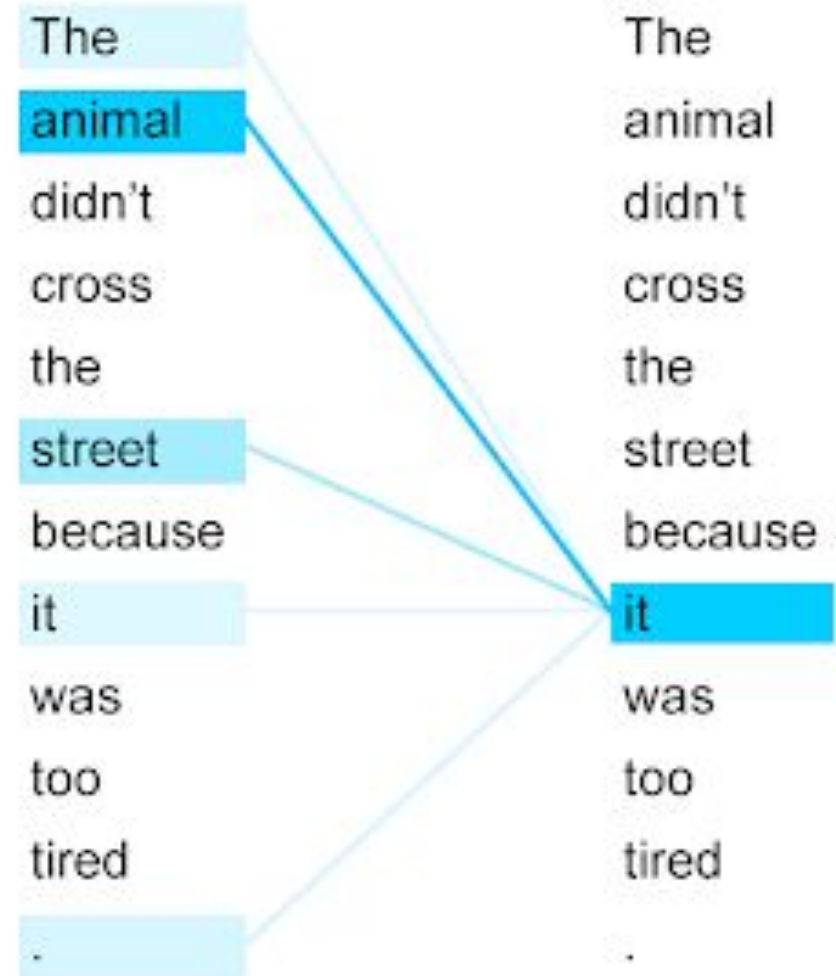
Self-Attention

Self-attention:

- every element in sequence can influence every other element
- learn weighting (“attention”) for each pair of elements

Attention: learnable pairwise weighting that depends on other sequence

Self-Attention: use input sequence for attention



Self-Attention: Motivation

Sequence prediction with pairwise transition probabilities.

Toy example: predict word after *ran*:

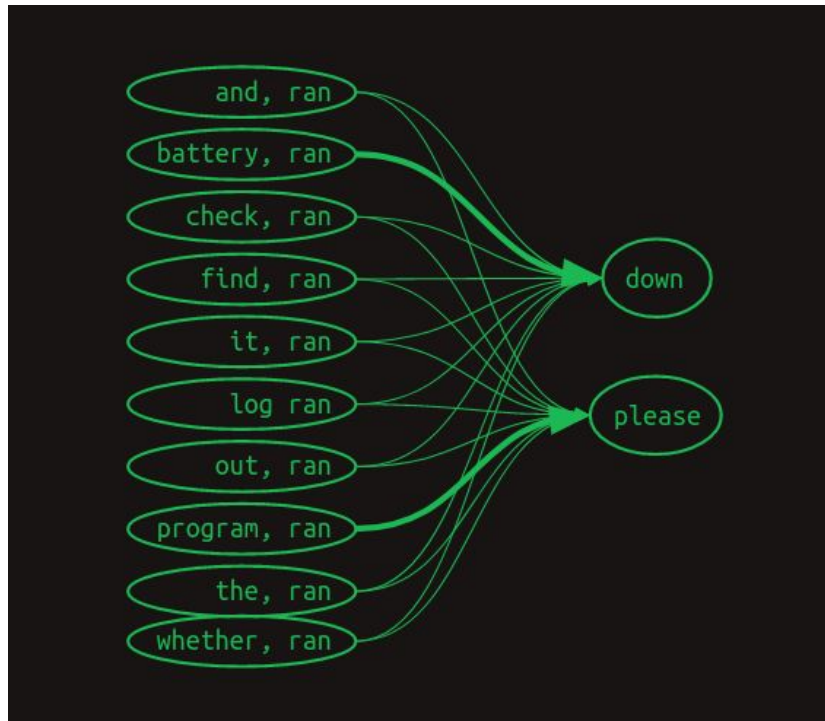
- *Check the program log and find out whether it ran please.*
- *Check the battery log and find out whether it ran down please.*

Self-Attention: Motivation

Sequence prediction with pairwise transition probabilities.

Toy example: predict word after *ran*:

- *Check the program log and find out whether it ran please.*
- *Check the battery log and find out whether it ran down please.*



Pairwise
transitions

Pairwise transition probabilities

	and	battery	check	down	find	it	log	out	please	program	ran	the	whether
and, ran				.5					.5				
battery, ran				1					0				
check, ran				.5					.5				
down, ran													
find, ran				.5					.5				
it, ran				.5					.5				
log, ran				.5					.5				
out, ran				.5					.5				
please, ran													
program, ran				0					1				
ran, ran													
the, ran				.5					.5				
whether, ran				.5					.5				

Self-Attention: Motivation

Sequence prediction with pairwise transition probabilities.

Toy example: predict word after *ran*:

- *Check the program log and find out whether it ran please.*
- *Check the battery log and find out whether it ran down please.*

Only two pairs matter!

Pairwise transition probabilities

	and	battery	check	down	find	it	log	out	please	program	ran	the	whether
and, ran			.5					.5					
battery, ran		1						0					
check, ran			.5					.5					
down, ran													
find, ran			.5					.5					
it, ran			.5					.5					
log, ran			.5					.5					
out, ran			.5					.5					
please, ran													
program, ran			0					1					
ran, ran													
the, ran			.5					.5					
whether, ran			.5					.5					

Self-Attention: Motivation

Sequence prediction with pairwise transition probabilities.

Toy example: predict word after *ran*:

- Check the program log and find out whether it ran please.
- Check the battery log and find out whether it ran down please.

-> Masking: keep only informative transitions



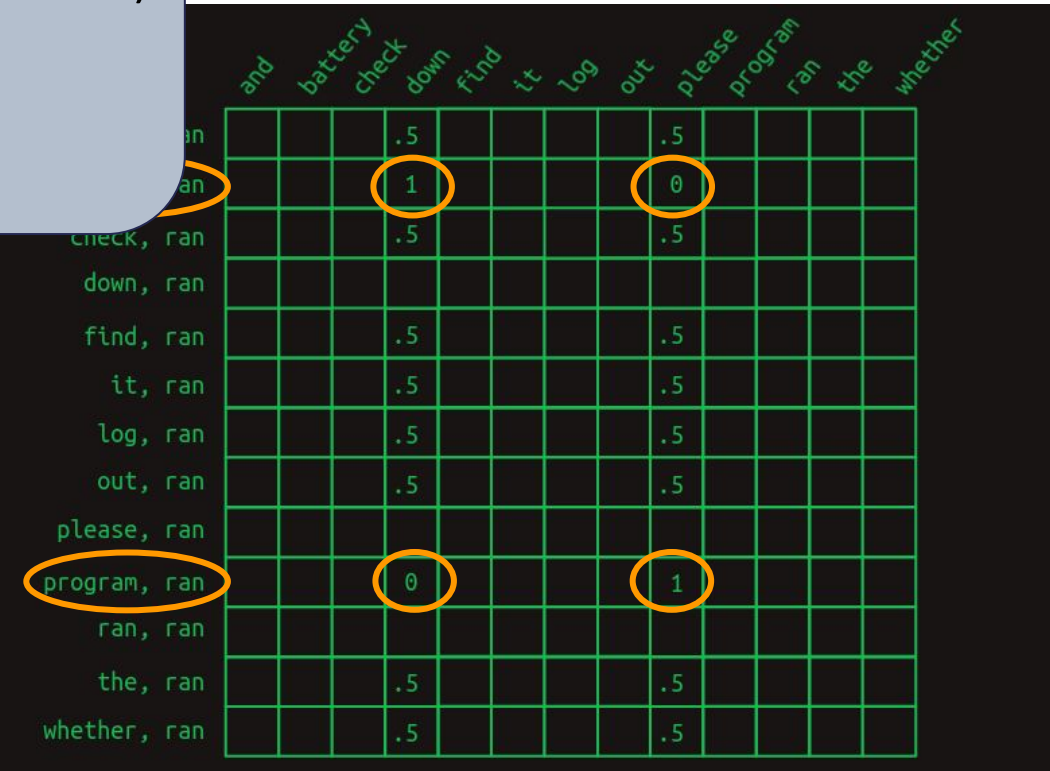
Pairwise transition probabilities

	and	battery	check	down	find	it	log	out	please	program	ran	the	whether
and, ran				.5					.5				
battery, ran			1						0				
check, ran			.5						.5				
down, ran													
find, ran			.5						.5				
it, ran			.5						.5				
log, ran			.5						.5				
out, ran			.5						.5				
please, ran													
program, ran			0						1				
ran, ran													
the, ran			.5						.5				
whether, ran			.5						.5				

Self-Attention: Motivation

Goal of self attention / transformers:
express transition probabilities with soft masking (= attention)
s.t. can be learned by neural network layer

Pairwise transition probabilities



Self-Attention

Input: represent by word embedding
(= vector per word)

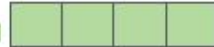
Need 3 vectors for each element to model
attention:

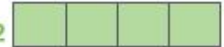
Input

Thinking

Machines

Embedding

X_1 

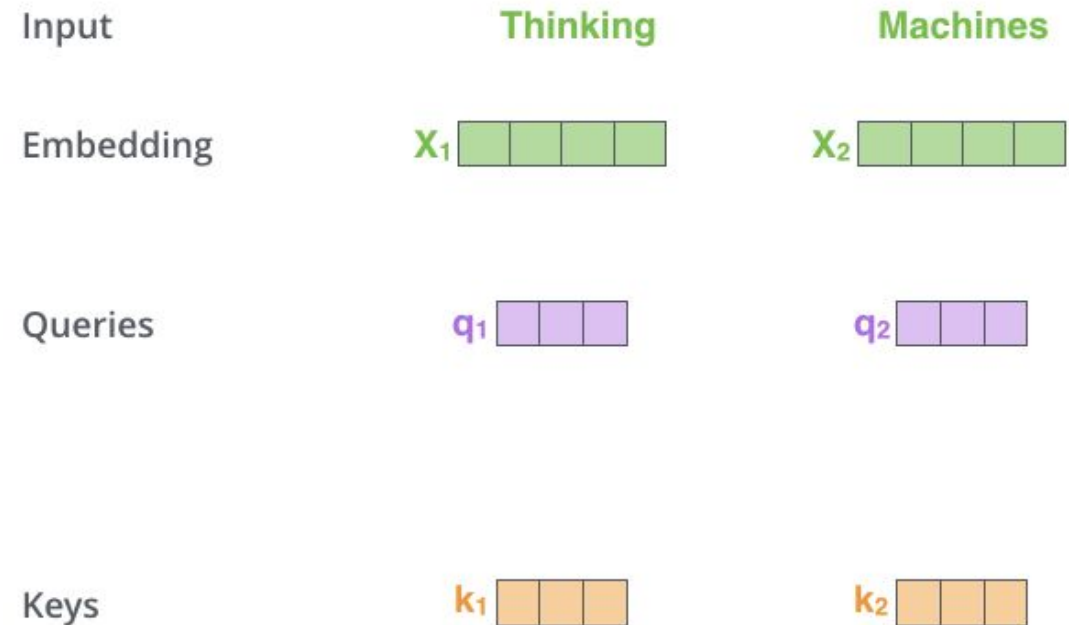
X_2 

Self-Attention

Input: represent by word embedding
(= vector per word)

Need 3 vectors for each element to model
attention:

- Queries $\mathbf{q}$, keys $\mathbf{k}$ for attention:
 $\mathbf{q}$ encodes current element
 $\mathbf{k}$ encodes potential masks



Self-Attention

Input: represent by word embedding
(= vector per word)

Need 3 vectors for each element to model
attention:

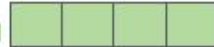
- Queries $\mathbf{q}$, keys $\mathbf{k}$ for attention:
 $\mathbf{q}$ encodes current element
 $\mathbf{k}$ encodes potential masks
- Values $\mathbf{v}$: pairwise transitions / weights

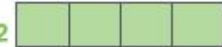
Input

Thinking

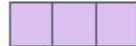
Machines

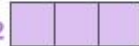
Embedding

$\mathbf{x}_1$ 

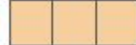
$\mathbf{x}_2$ 

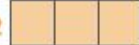
Queries

$\mathbf{q}_1$ 

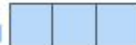
$\mathbf{q}_2$ 

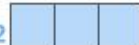
Keys

$\mathbf{k}_1$ 

$\mathbf{k}_2$ 

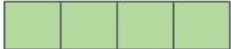
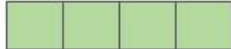
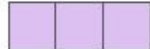
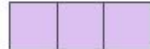
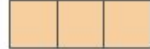
Values

$\mathbf{v}_1$ 

$\mathbf{v}_2$ 

Self-Attention

Query, Key: compute attention:

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

d_k : dimension of q, k, v
 = vector dim of transformed
 sequence, hyperparameter

Self-Attention

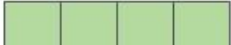
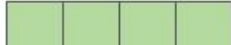
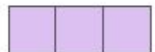
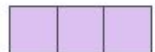
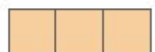
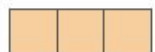
Query, Key: compute attention:

Q: Matrix of query vectors
 $sequence_length (s_l) \times d_k$

K: Matrix of keys
 same dim as **Q**

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

d_k : dimension of q, k, v

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self-Attention

Query, Key: compute attention:

Q: Matrix of query vectors
 $s_l \times d_k$

K: Matrix of keys
 same dim as **Q**

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

applied per row

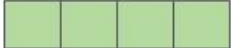
d_k : dimension of q, k, v

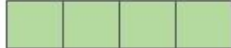
Input

Thinking

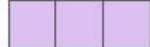
Machines

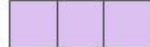
Embedding

x_1 

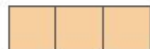
x_2 

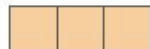
Queries

q_1 

q_2 

Keys

k_1 

k_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Self-Attention

Query, Key: compute attention:

Q: Matrix of query vectors
 $s_l \times d_k$

K: Matrix of keys
 same dim as **Q**

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

applied per row

Matrix-matrix multi.:

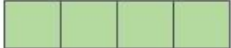
$$(s_l \times d_k) * (d_k \times s_l) = (s_l \times s_l)$$

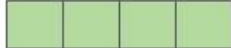
Input

Thinking

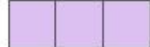
Machines

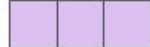
Embedding

x_1 

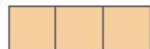
x_2 

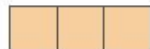
Queries

q_1 

q_2 

Keys

k_1 

k_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Self-Attention

Query, Key: compute attention:

Q: Matrix of query vectors
 $s_l \times d_k$

K: Matrix of keys
 same dim as Q

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

applied per row

Matrix-matrix multi.:

$$(s_l \times d_k) * (d_k \times s_l) = (s_l \times s_l)$$

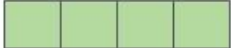
soft mask for each pair of elements

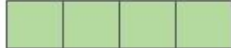
Input

Thinking

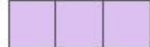
Machines

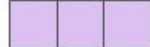
Embedding

x_1 

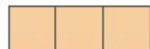
x_2 

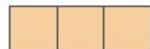
Queries

q_1 

q_2 

Keys

k_1 

k_2 

Score

$q_1 \cdot k_1 = 112$

$q_1 \cdot k_2 = 96$

Divide by 8 ($\sqrt{d_k}$)

14

12

Softmax

0.88

0.12

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

V: Matrix of values
 same dim as **Q, K**

Input

Embedding

Queries

Keys

Values

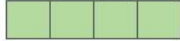
Score

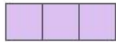
Divide by 8 ($\sqrt{d_k}$)

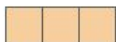
Softmax

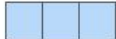
Softmax
 X
 Value

Thinking

x₁ 

q₁ 

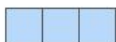
k₁ 

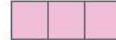
v₁ 

q₁ • k₁ = 112

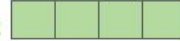
14

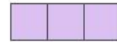
0.88

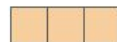
v₁ 

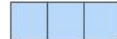
z₁ 

Machines

x₂ 

q₂ 

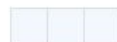
k₂ 

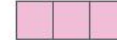
v₂ 

q₁ • k₂ = 96

12

0.12

v₂ 

z₂ 

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

V: Matrix of values
same dim as **Q, K**

Matrix-matrix multipl.:

$$(s_l \times s_l) * (s_l \times d_k) = (s_l \times d_k)$$

Input

Embedding

Queries

Keys

Values

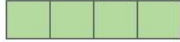
Score

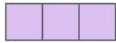
Divide by 8 ($\sqrt{d_k}$)

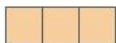
Softmax

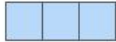
Softmax
X
Value

Thinking

x₁ 

q₁ 

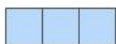
k₁ 

v₁ 

q₁ • k₁ = 112

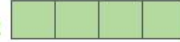
14

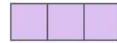
0.88

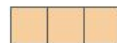
v₁ 

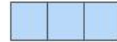
z₁ 

Machines

x₂ 

q₂ 

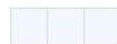
k₂ 

v₂ 

q₂ • k₂ = 96

12

0.12

v₂ 

z₂ 

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

V: Matrix of values
same dim as Q, K

Matrix-matrix multipl.:

$$(s_l \times s_l) * (s_l \times d_k) = (s_l \times d_k)$$

Transformed sequence! (same length)

Input

Embedding

Queries

Keys

Values

Score

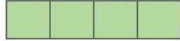
Divide by 8 ($\sqrt{d_k}$)

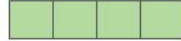
Softmax

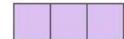
Softmax
X
Value

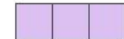
Thinking

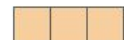
Machines

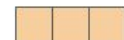
x_1 

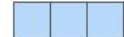
x_2 

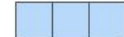
q_1 

q_2 

k_1 

k_2 

v_1 

v_2 

$q_1 \cdot k_1 = 112$

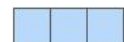
$q_1 \cdot k_2 = 96$

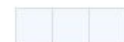
14

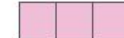
12

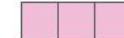
0.88

0.12

v_1 

v_2 

z_1 

z_2 

Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

V : Matrix of values
same dim as Q, K

Matrix-matrix multipl.:

$$(s_l \times s_l) * (s_l \times d_k) = (s_l \times d_k)$$

Transformed sequence! (same length)

The diagram illustrates the Self-Attention mechanism. It shows the calculation of the attention matrix Z as the product of Q and K^T , followed by a softmax operation, and then the final output V .

Matrix dimensions are indicated by the number of rows and columns in the grid representations:

- Q (purple grid) is 3×3 .
- K^T (orange grid) is 3×3 .
- V (blue grid) is 3×3 .
- The result Z (pink grid) is 3×3 .

The operation is shown as:

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

Where is the learning happening???

Where is the learning happening???

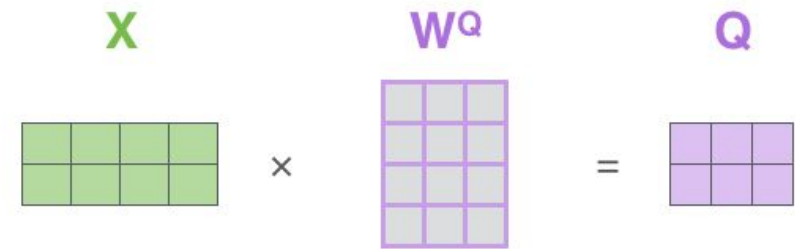
How do we obtain **Q**, **K**, **V**?

Where is the learning happening???

How do we obtain **Q**, **K**, **V**?

Computed from **X** (input tokens) with *learned* weights:

- Fully connected layer for **Q**, **K**, **V**



Self-Attention: Dimensions and Parameters

Input (**X**) / output (**Z**):

- $s_l \times d_i$, $s_l \times d_k$

Dimensions of **Q**, **K**, **V**:

- $s_l \times d_k$, $s_l \times d_k$, $s_l \times d_k$

Dimensions of **W_q**, **W_k**, **W_v**:

- $d_i \times d_k$, $d_i \times d_k$, $d_i \times d_k$

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = X W_q$$

$$K = X W_k$$

$$V = X W_v$$

Self-Attention: Dimensions and Parameters

Input (**X**) / output (**Z**):

- $s_l \times d_i$, $s_l \times d_k$

Dimensions of **Q**, **K**, **V**:

- $s_l \times d_k$, $s_l \times d_k$, $s_l \times d_k$

Dimensions of **W_q**, **W_k**, **W_v**:

- $d_i \times d_k$, $d_i \times d_k$, $d_i \times d_k$

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = X W_q$$

$$K = X W_k$$

$$V = X W_v$$

Questions:

- What are the learnable parameters?
- Is the sequence length fixed?

Self-Attention: Dimensions and Parameters

Input (**X**) / output (**Z**):

- $s_l \times d_i$, $s_l \times d_k$

Dimensions of **Q**, **K**, **V**:

- $s_l \times d_k$, $s_l \times d_k$, $s_l \times d_k$

Dimensions of **W_q**, **W_k**, **W_v**:

- $d_i \times d_k$, $d_i \times d_k$, $d_i \times d_k$

$$Z = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = X W_q$$

$$K = X W_k$$

$$V = X W_v$$

Questions:

- What are the learnable parameters?
 - Weights of **W_q**, **W_k**, **W_v**
- Is the sequence length fixed?
 - No! learnable params don't depend on seq. length!

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = X W_q$$

$$K = X W_k$$

$$V = X W_v$$

$$\text{Out} = \text{activation}(W X)$$

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Example: $s_i = 512$, $d_i = d_k = 768$

Attention: **How many parameters?**

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$\begin{aligned} Q &= XW_q \\ K &= XW_k \\ V &= XW_v \end{aligned}$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Example: $s_i = 512$, $d_i = d_k = 768$

Attention: **How many parameters?**

$$3 * d_i * d_k = 3 * 768^2 \sim 1.7 \text{ million}$$

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Example: $s_i = 512$, $d_i = d_k = 768$

Attention: How many parameters?

$$3 * d_i * d_k = 3 * 768^2 \sim 1.7 \text{ million}$$

Fully-connected: **How many parameters?**

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Example: $s_i = 512$, $d_i = d_k = 768$

Attention: How many parameters?

$$3 * d_i * d_k = 3 * 768^2 \sim 1.7 \text{ million}$$

Fully-connected: **How many parameters?**

$$(s_i * d_i) * (s_i * d_k) \sim 154 \text{ billion}$$

Self-Attention vs. Fully-Connected

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

**Learnable
parameters**

$$\text{Out} = \text{activation}(WX)$$

**Learnable
parameters**

Example: $s_i = 512$, $d_i = d_k = 768$

Attention: How many parameters?

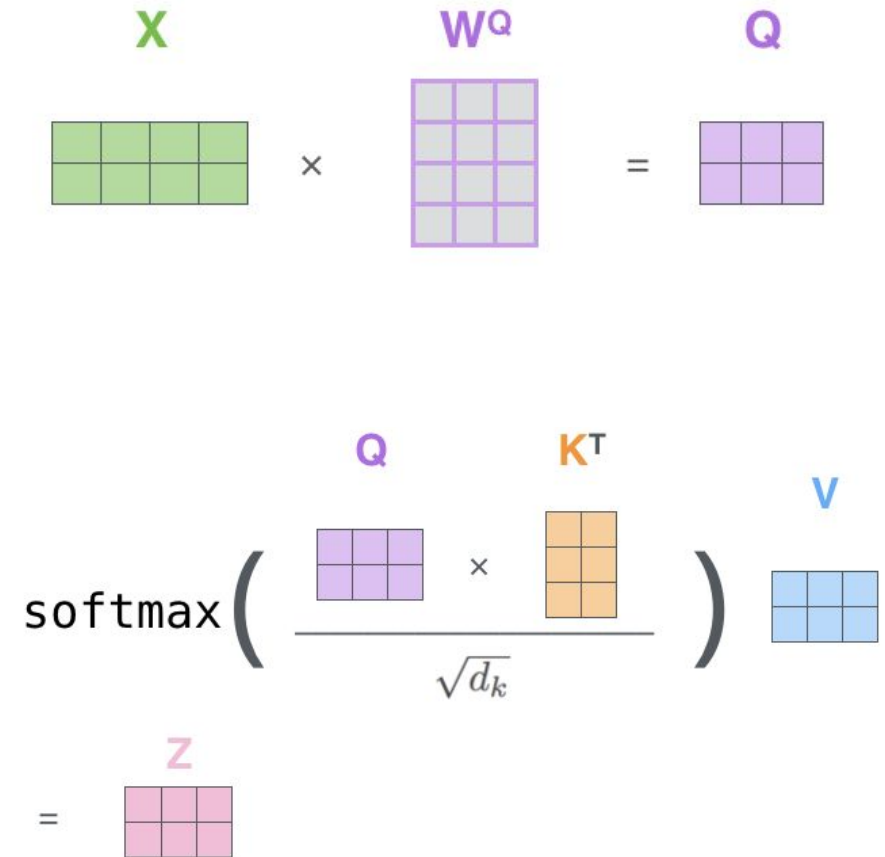
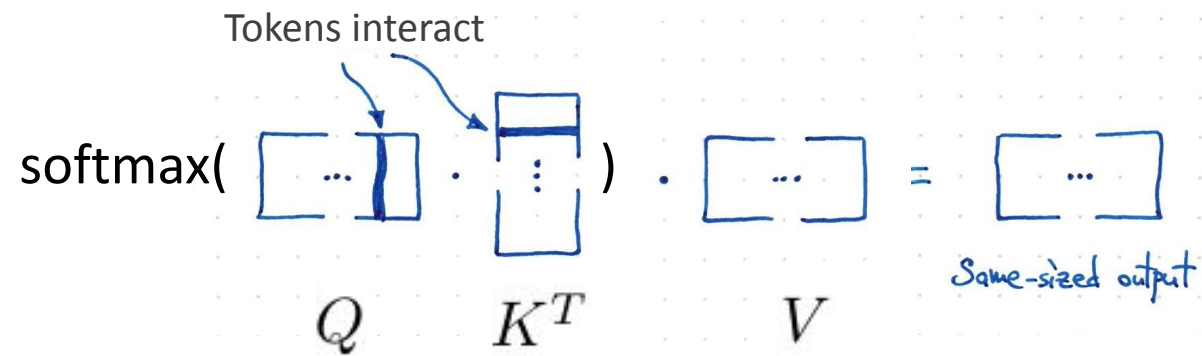
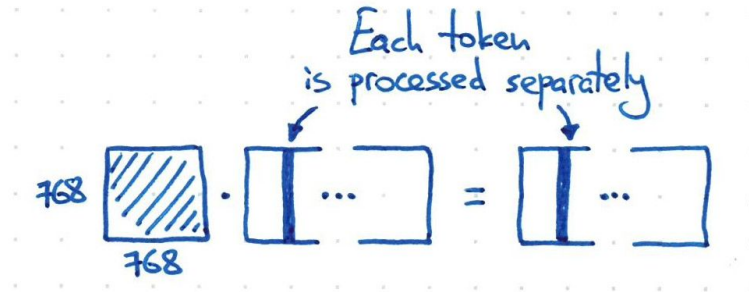
$$3 * d_i * d_k = 3 * 768^2 \sim 1.7 \text{ million}$$

Fully-connected: **How many parameters?**

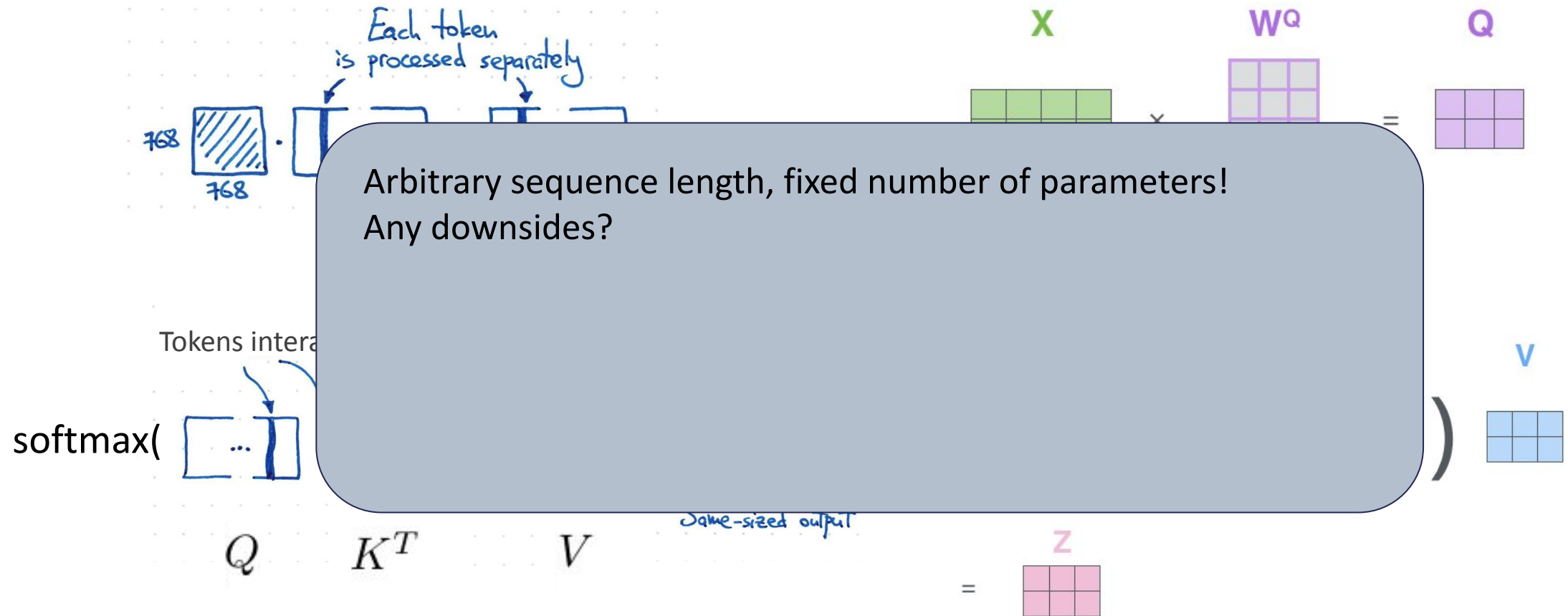
$$(s_i * d_i) * (s_i * d_k) \sim 154 \text{ billion}$$

Can only process fixed seq. length!

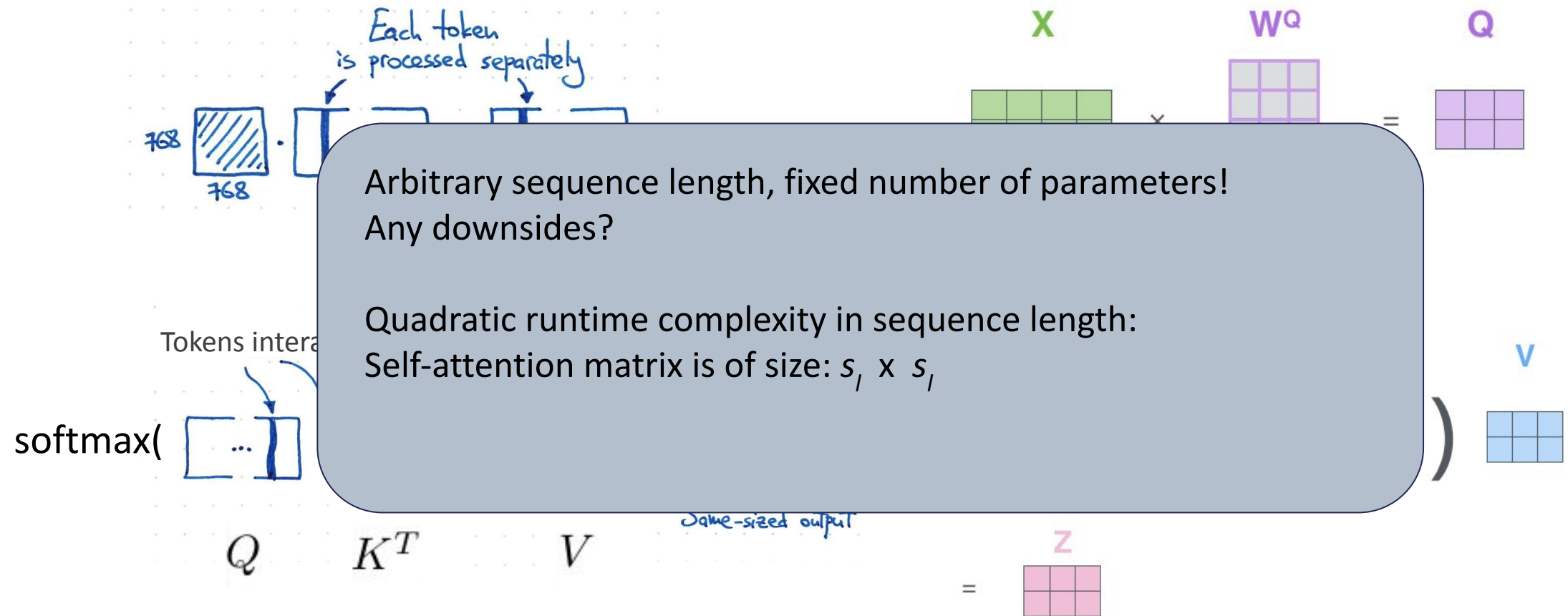
Self-attention: overview



Self-attention: overview



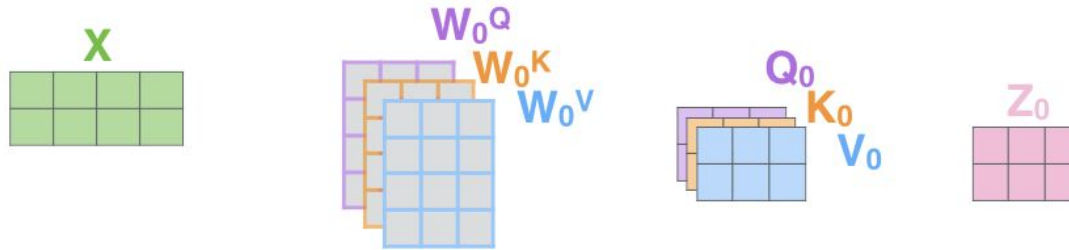
Self-attention: overview



Multi-headed self-attention

Softmax

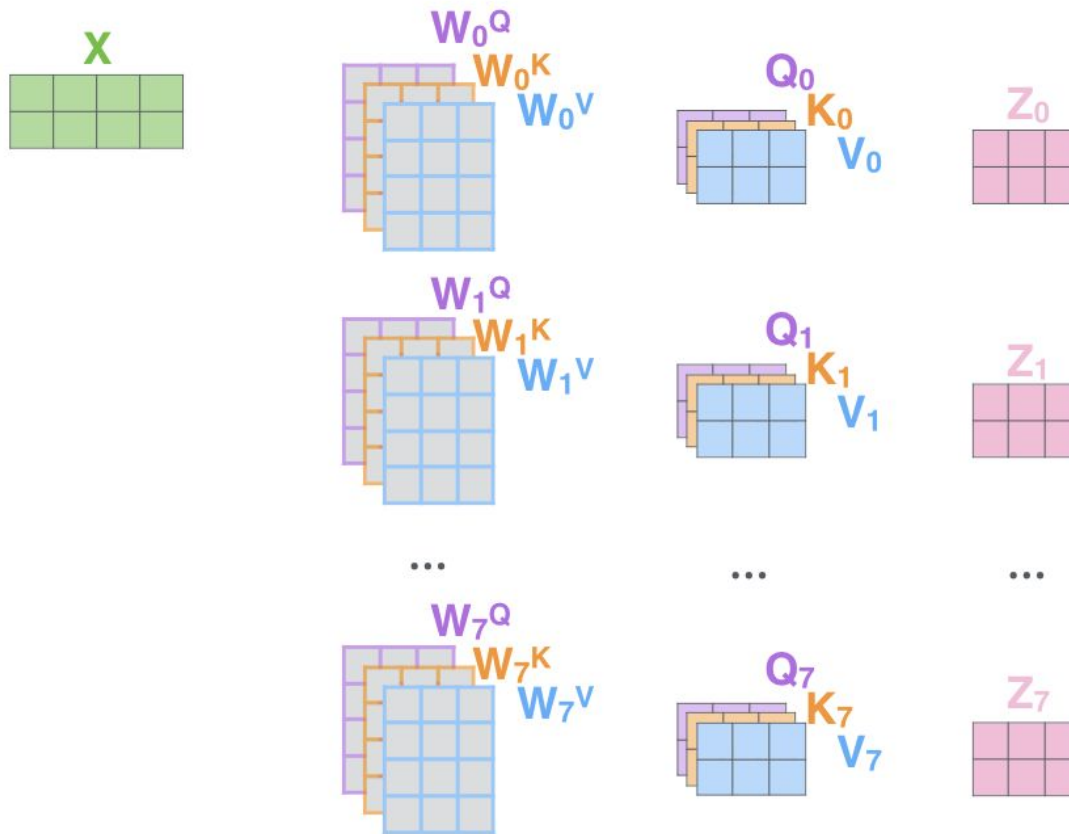
- Only one sequence pair with high value per element
- Focuses on a single pairwise interaction
-> limits modeling capability



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-headed self-attention

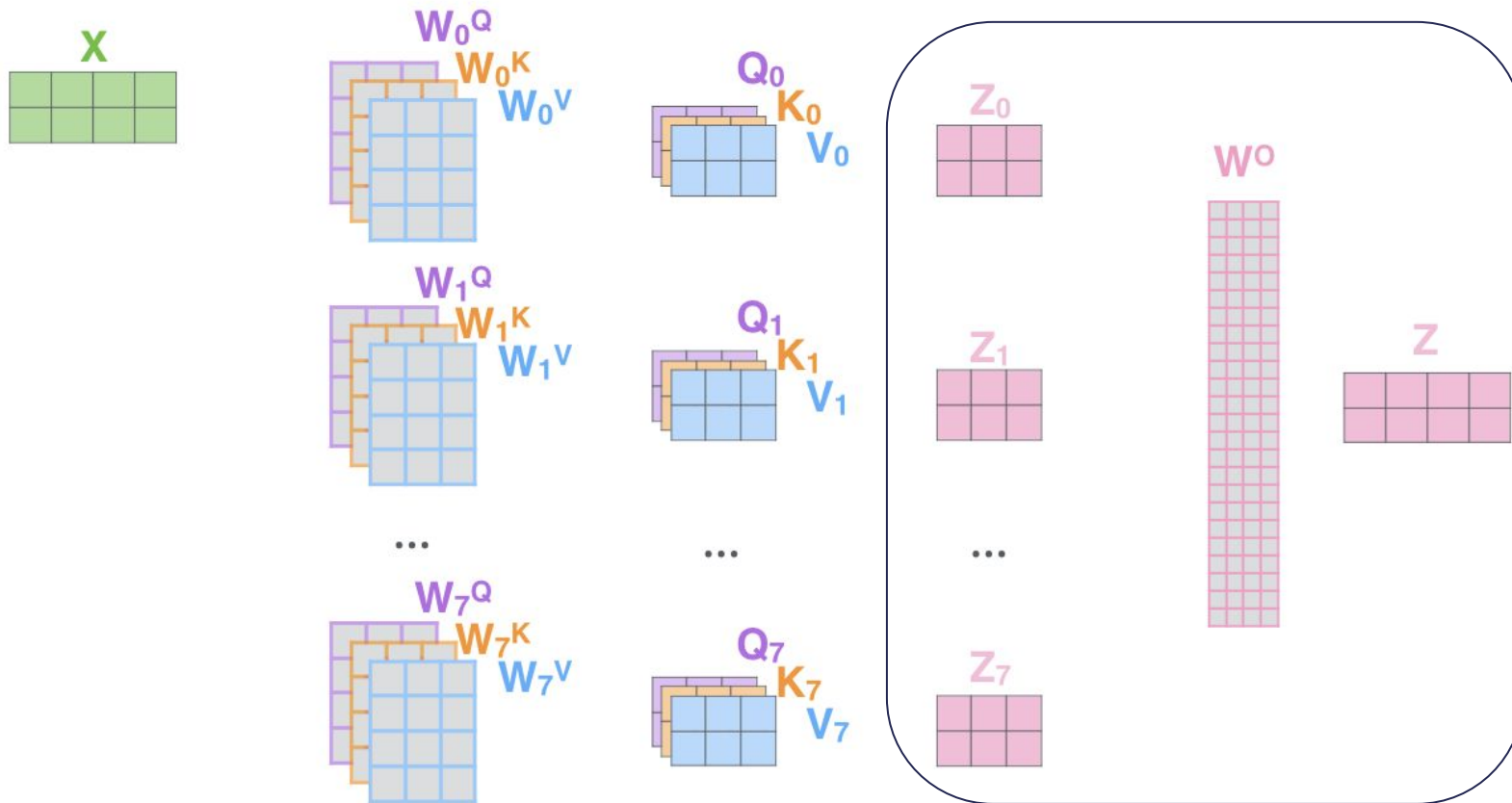
Multiple attention heads for
increased model capacity



Multi-headed self-attention

Multiple attention heads for
increased model capacity

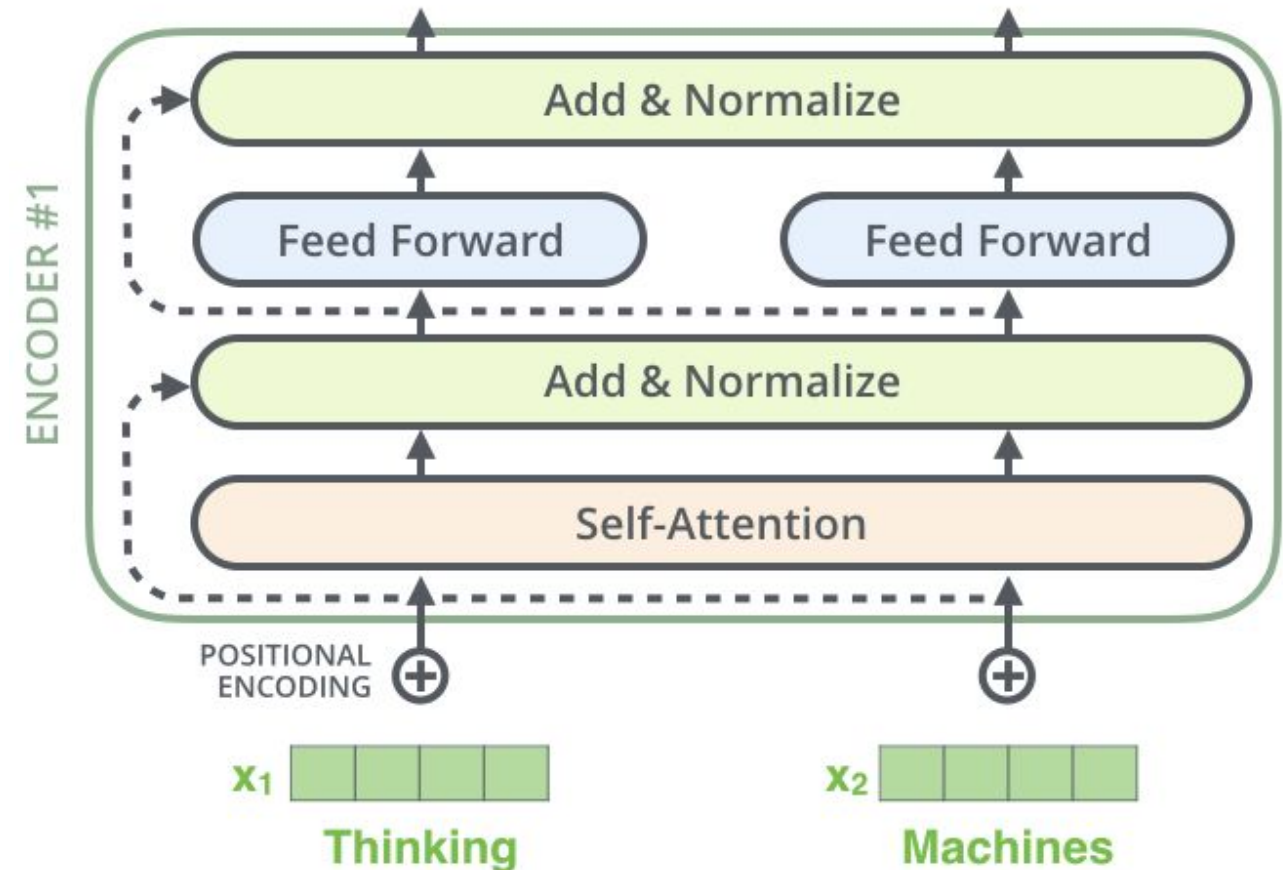
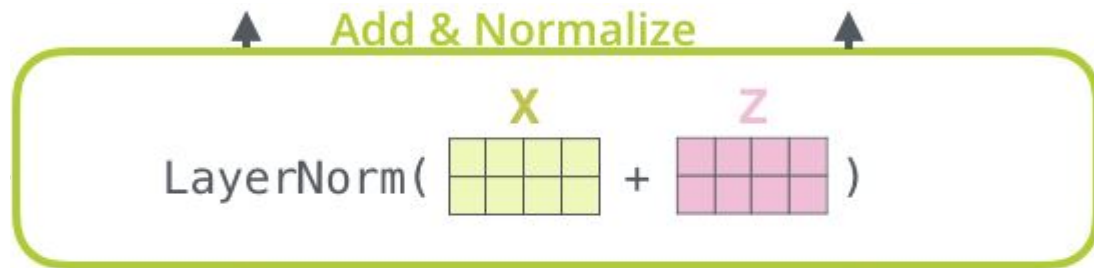
Combine with additional
feed-forward layer



Concat z_i outputs, project to z
with learned weight matrix

Transformer Block

Combine Self-Attention with residual connections & normalization



Input Embeddings

Word to embedding: *Tokenization*: translate to semantic vector space

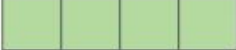


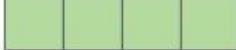
Input Embeddings

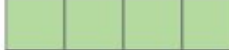
Word to embedding: *Tokenization*: translate to semantic vector space

Tokenization: own research topic in NLP
Popular method: word2vec

EMBEDDINGS

x_1 

x_2 

x_3 

INPUT

Je

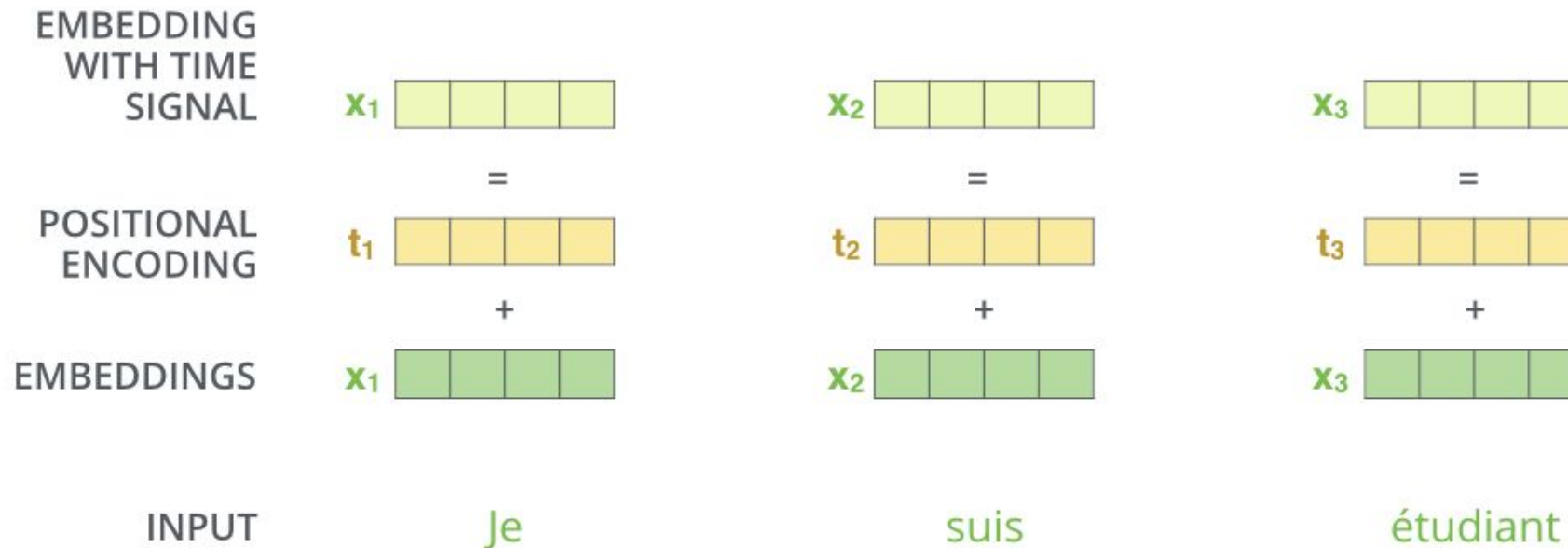
suis

étudiant

Input Embeddings

Word to embedding: *Tokenization*: translate to semantic vector space

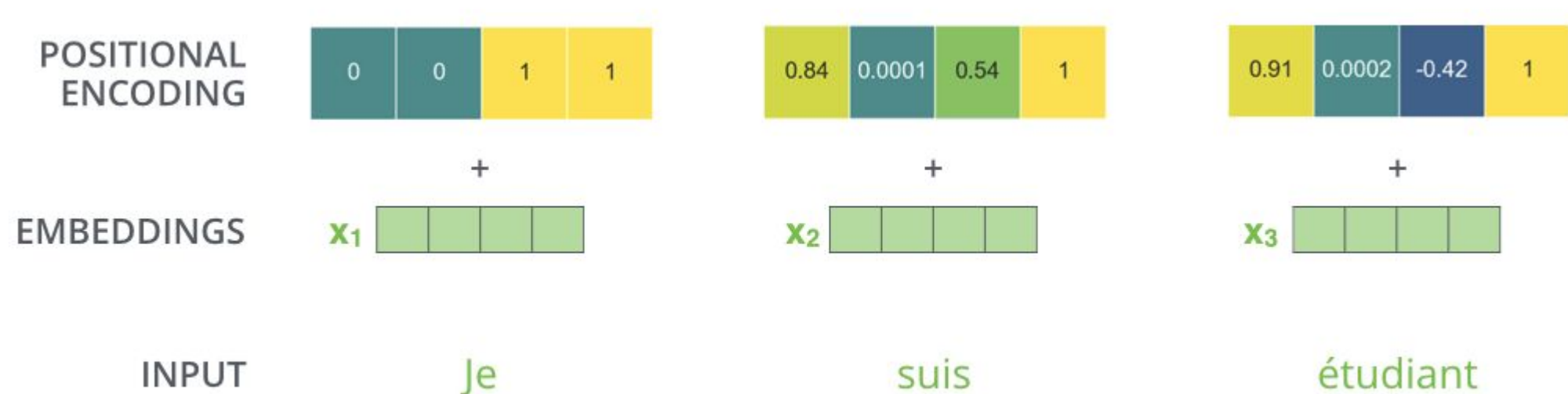
Positional embeddings: encode position of element in sequence (Trigonometric function of time)



Input Embeddings

Word to embedding: *Tokenization*: translate to semantic vector space

Positional embeddings: encode position of element in sequence (Trigonometric function of time)



Transformers in NLP

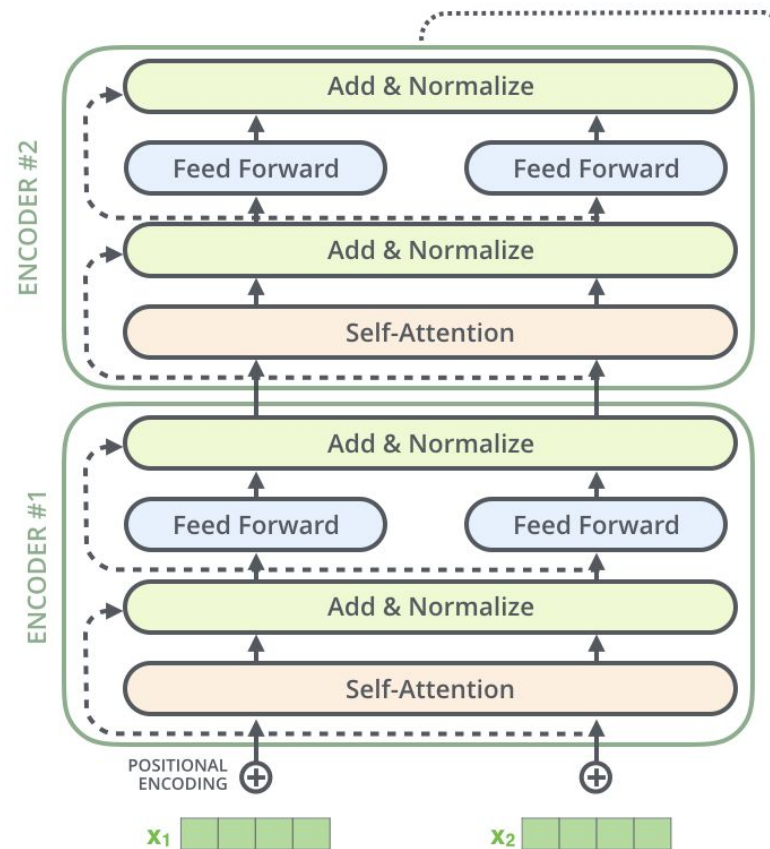
& Large Language Models

Translation with Transformers

First transformer application:

Translation

- Encoder: Transformer blocks.
Input sequence to embedding

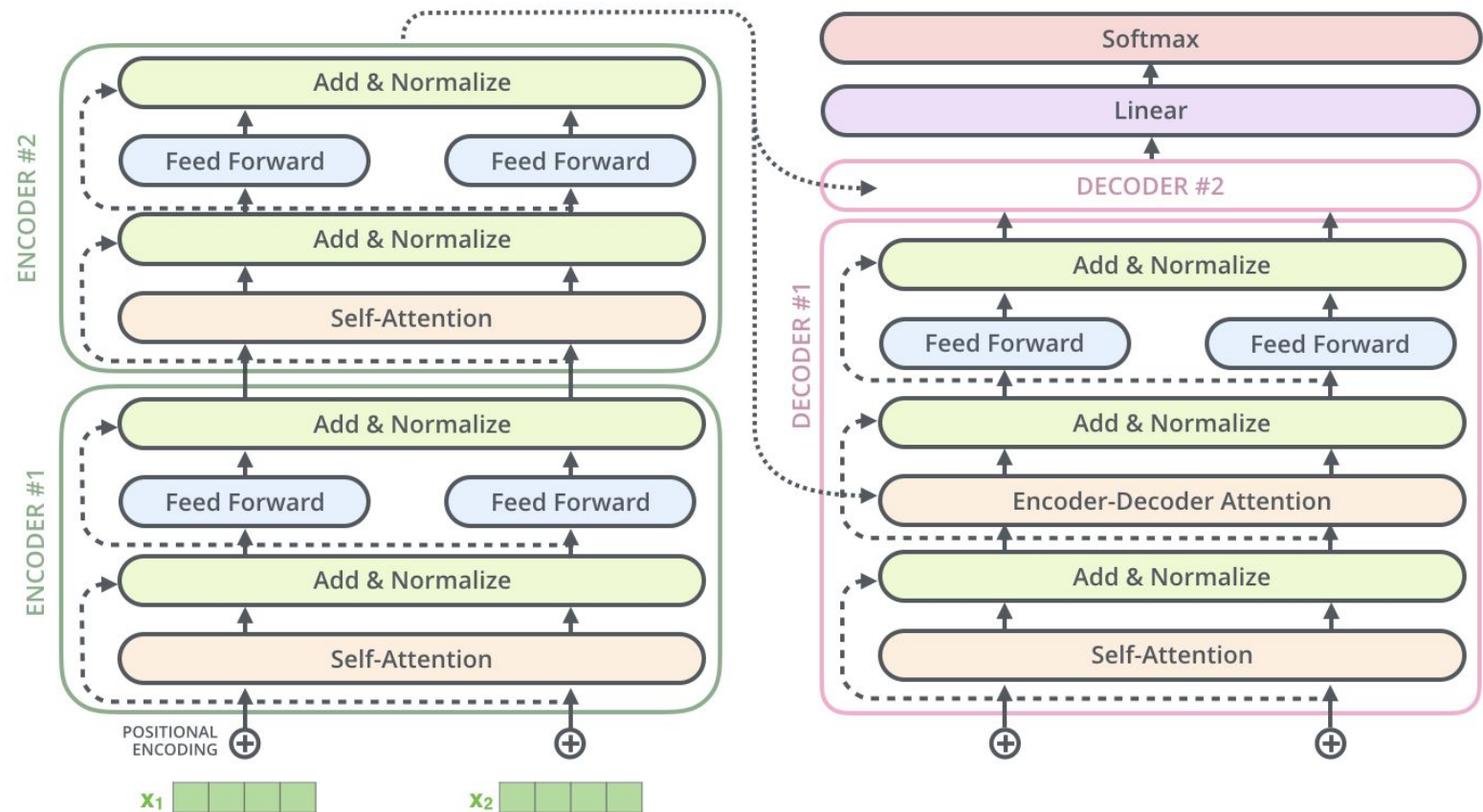


Translation with Transformers

First transformer application:

Translation

- Encoder: Transformer blocks. Input sequence to embedding
- Decoder: Transformer blocks. Latent space to output



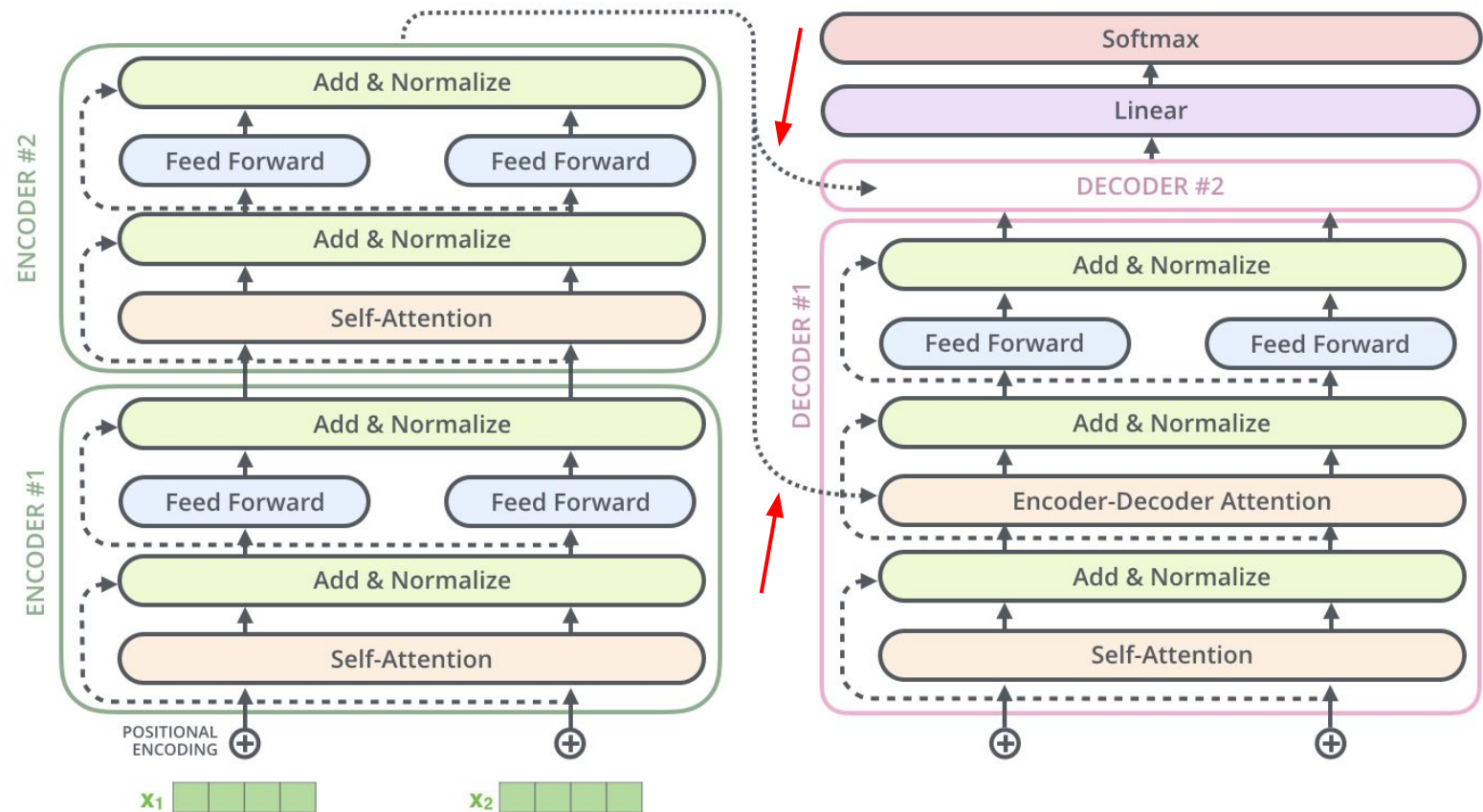
Translation with Transformers

First transformer application:

Translation

- Encoder: Transformer blocks. Input sequence to embedding
- Decoder: Transformer blocks. Latent space to output

Encoder-Decoder Attention / Cross Attention:
use keys and values from last encoder block in decoder



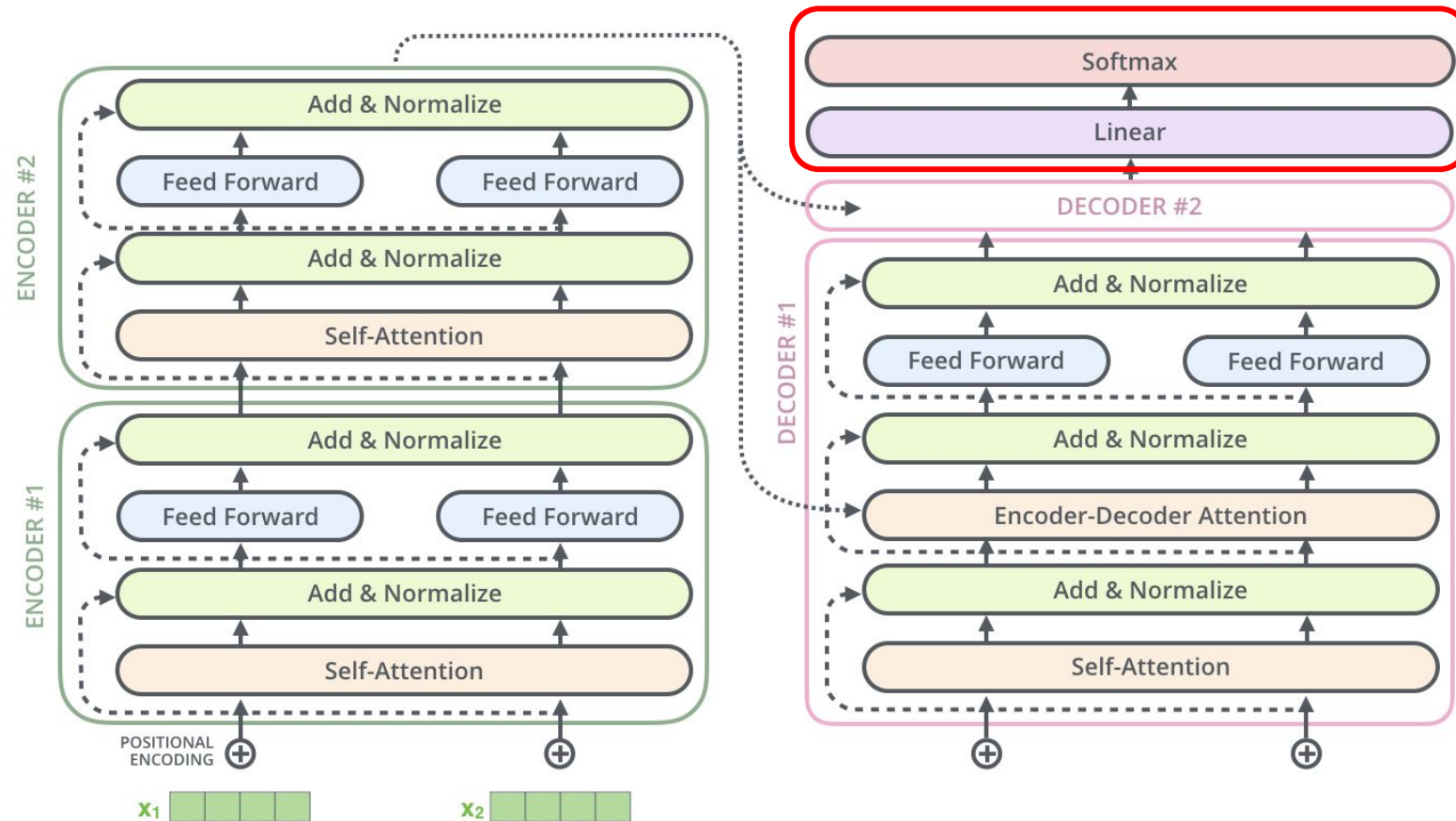
Translation with Transformers

First transformer application:

Translation

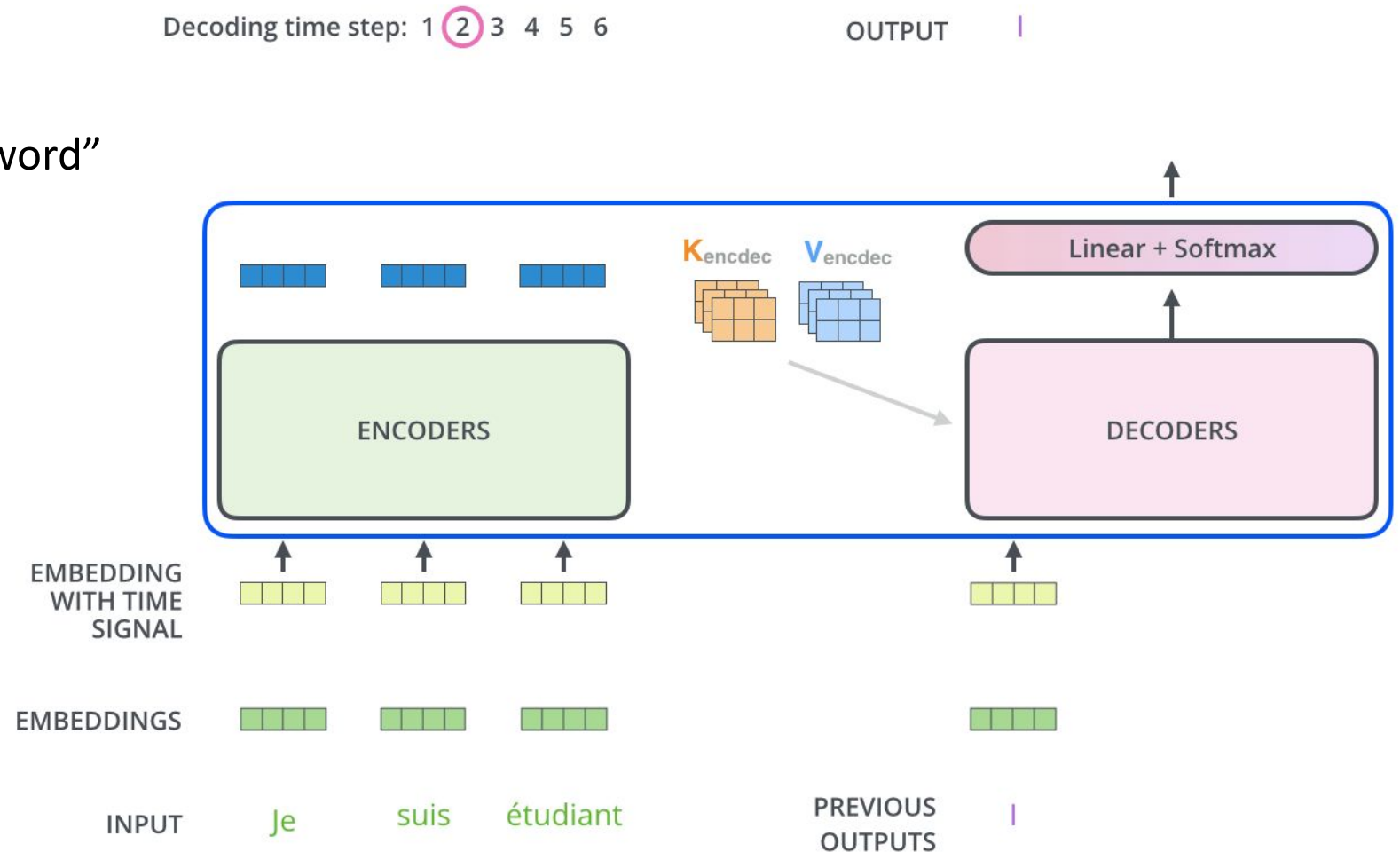
- Encoder: Transformer blocks. Input sequence to embedding
- Decoder: Transformer blocks. Latent space to output

Output layer:
one-hot encoding of all words
in vocabulary



Translation with Transformers

- Encode input “en block”
- Decode output “word-by-word”
with prev output as input



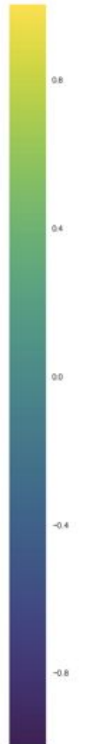
Translation with Transformers

Training: maximize cross entropy of output word probabilities on training data

Training data: texts with translation

Target Model Outputs

Output Vocabulary: a am I thanks student <eos>



Large language models

Transformers as chatbots or for search: less structured tasks than translation!

- No single correct answer.
- **How do we train?**

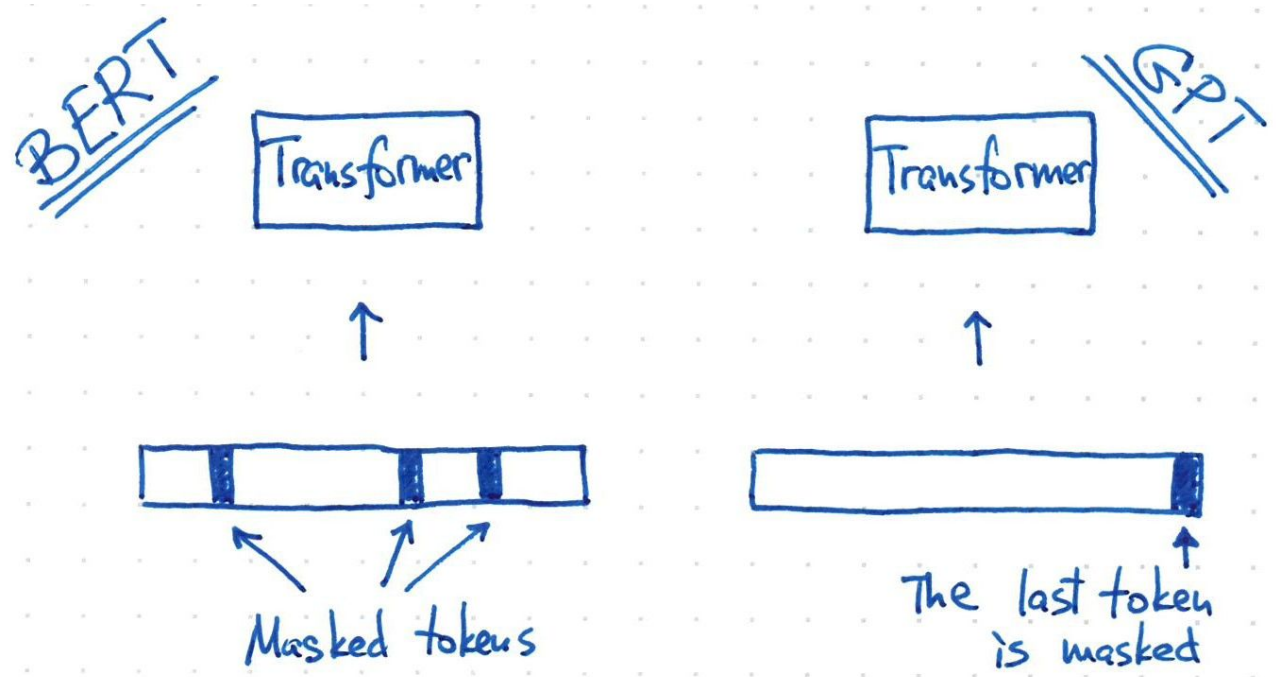
Large language models

BERT: <https://arxiv.org/abs/1810.04805>

GPT3: <https://arxiv.org/abs/2005.14165>

Transformers as chatbots or for search: less structured tasks than translation!

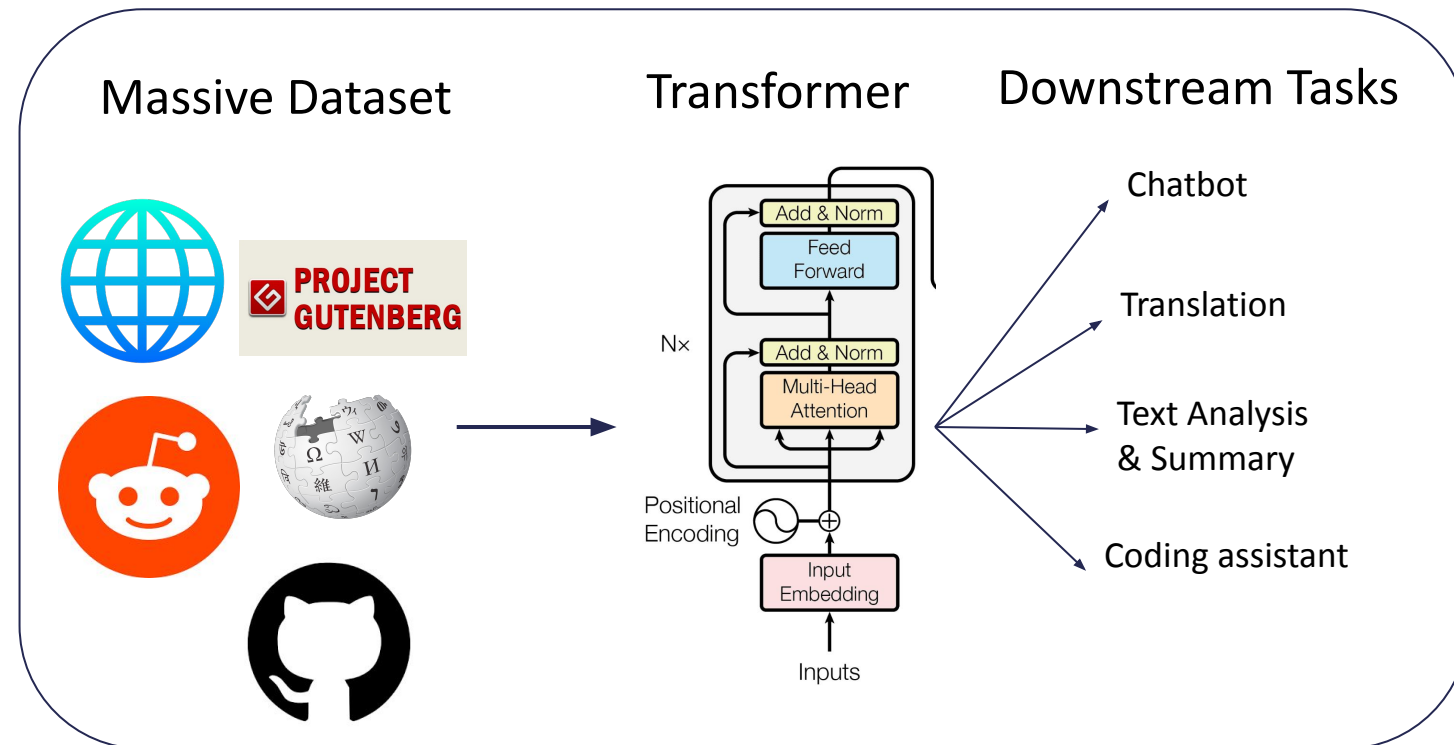
- No single correct answer.
- How do we train?
- **Unsupervised training:**
predict masked tokens from context.



Large language models

Building LLMs:

- Large transformer architecture
- Pretraining on massive text dataset
- Fine-tuning



Large language models

<https://arxiv.org/abs/2307.09288>

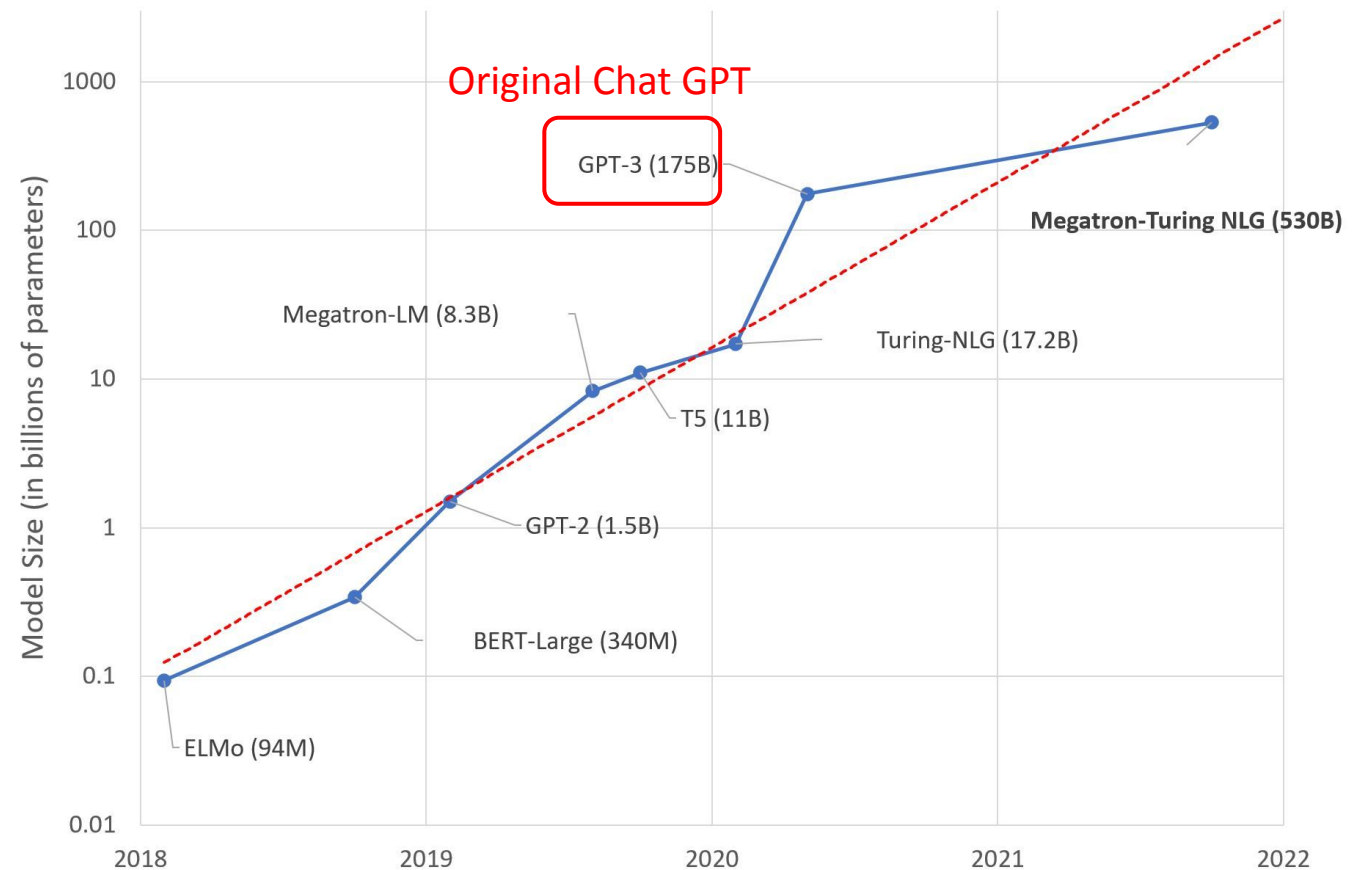
<https://huggingface.co/blog/large-language-models>

Building LLMs:

- **Large transformer architecture**
- Pretraining on massive text dataset
- Fine-tuning

Newer models:

- GPT4: 1.7 Trillion (not officially known)
- LLAMA 2 / 3: 7-70 Billion / 8-405 Billion (open source model)
- ...



Large language models

<https://arxiv.org/abs/2307.09288>

Building LLMs:

- Large transformer architecture
- **Pretraining on massive text dataset**
- Fine-tuning

LLAMA 2: 2 Trillion (10^{12}) token in train data

- ca. 2 Trillion words
- corresponds to ca. 20 Million books
(average 100,000 words per book)

In many cases: closed source and details on data are not known!

Large language models

<https://arxiv.org/abs/2307.09288>

Building LLMs:

- Large transformer architecture
- **Pretraining on massive text dataset**
- Fine-tuning

LLAMA 2: 2 Trillion (10^{12}) token in train data

- ca. 2 Trillion words
- corresponds to ca. 20 Million books (average 100,000 words per book)

Training needs GPU Cluster and is very expensive!

LLAMA 2: Cluster of A100 GPUs, largest model trained for almost 2 million single GPU hours.

		Time (GPU hours)	Power Consumption (W)	Carbon Emitted (tCO ₂ eq)
LLAMA 2	7B	184320	400	31.22
	13B	368640	400	62.44
	34B	1038336	350	153.90
	70B	1720320	400	291.42
Total		3311616		539.00

Per GPU

Large language models

Instruction Finetuning: <https://arxiv.org/abs/2203.02155>

RLHF: <https://arxiv.org/abs/1909.08593>

Building LLMs:

- Large transformer architecture
- Pretraining on massive text dataset
- **Fine-tuning:** to improve model for application (e.g. chat)
- Instruction fine-tuning (supervised):
 - Human examples to train model
 - Similar training objective to translation
- Reinforcement Learning from Human Feedback (RLHF)
 - Humans rank model outputs
 - Reward generated based on ranking
 - Model is trained to maximize reward

Large language models

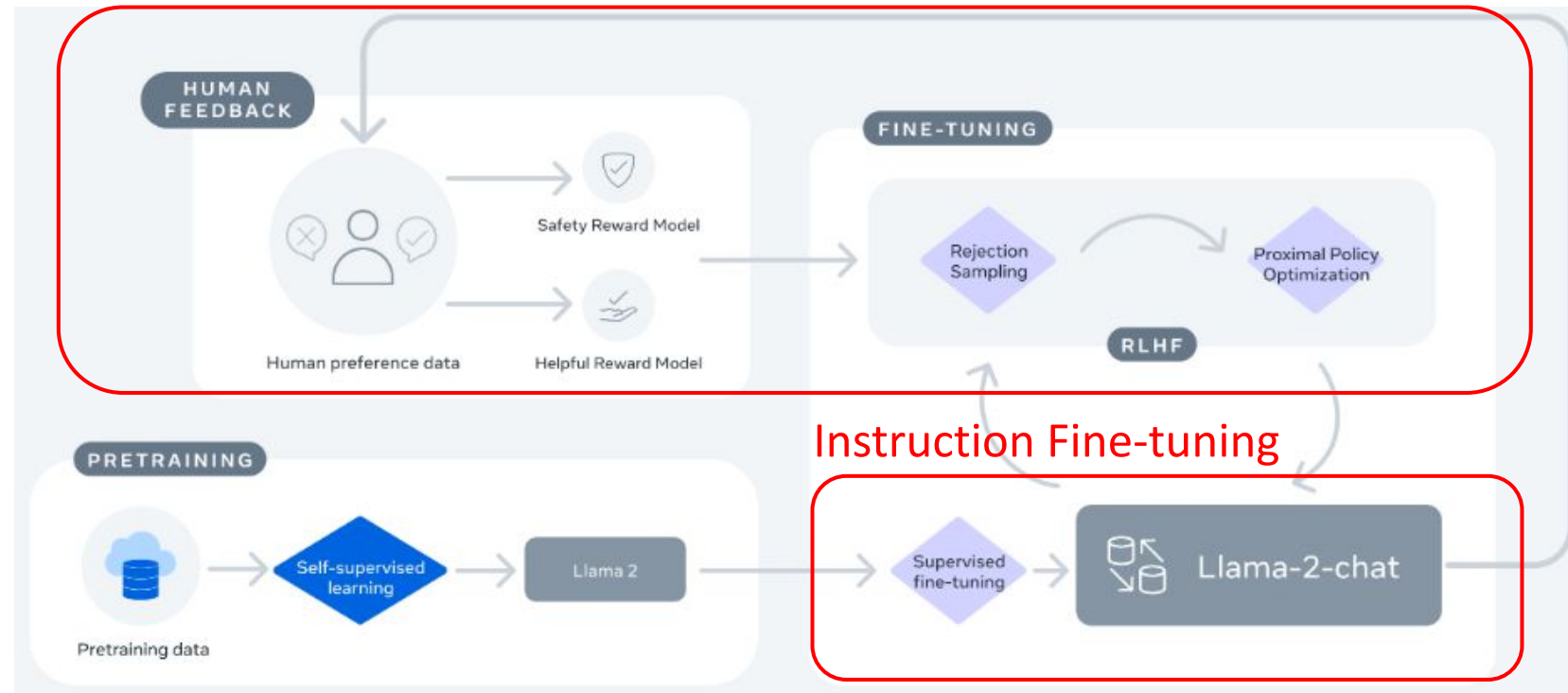
<https://arxiv.org/abs/2307.09288>

Building LLMs:

- Large transformer architecture
- Pretraining on massive text dataset
- **Fine-tuning:** to improve model for application (e.g. chat)

RLHF

LLAMA2:



Large language models

<https://arxiv.org/abs/2307.09288>

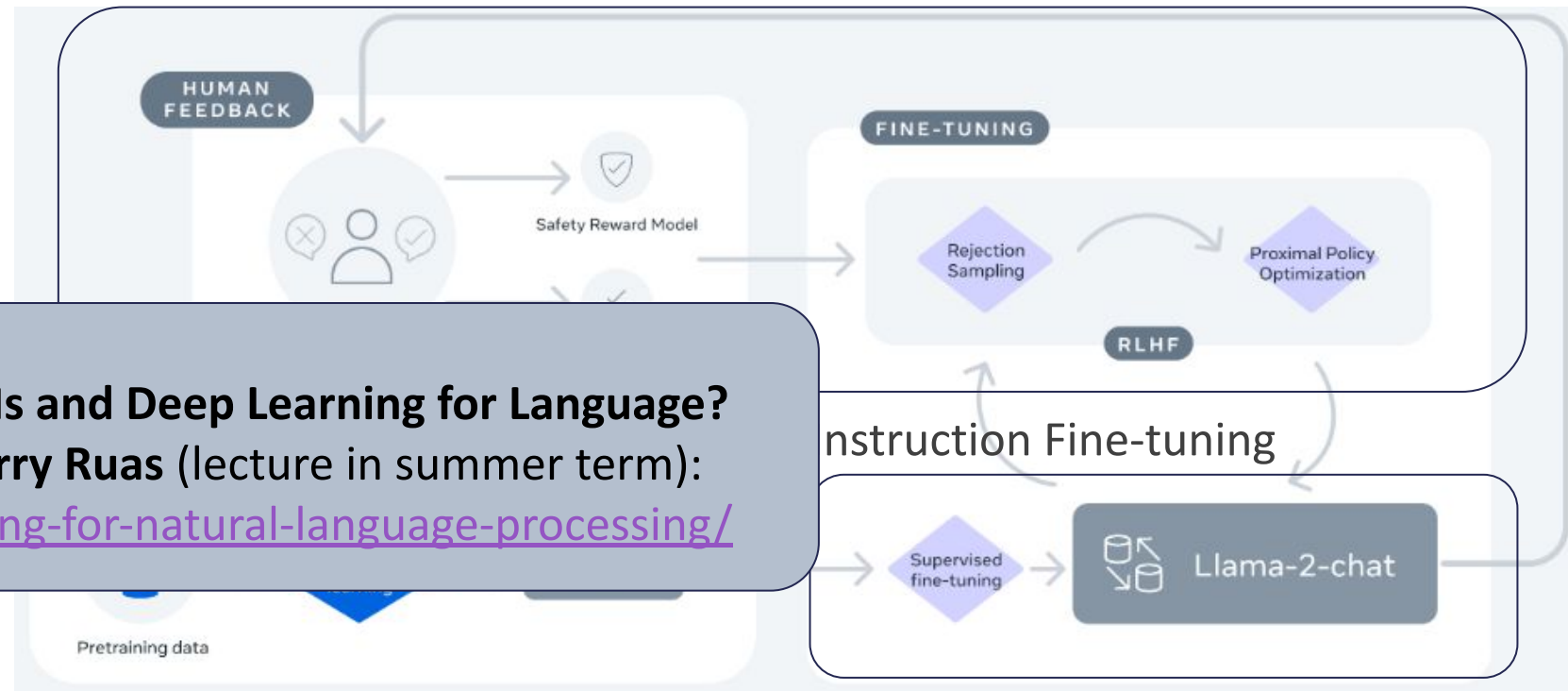
Building LLMs:

- Large transformer architecture
- Pretraining on massive text dataset
- **Fine-tuning:** to improve model for application (e.g. chat)

RLHF

LLAMA2:

Want to know more about LLMs and Deep Learning for Language?
DL for NLP by Bela Gipp and Terry Ruas (lecture in summer term):
<https://gipplab.org/deep-learning-for-natural-language-processing/>



Vision Transformer

Transformers for Vision

Why?

Very successful in NLP, esp. for large data! Maybe same for vision?

Problem: Image = 2D input, transformer expects 1D sequence

-> need tokenization for images

Transformers for Vision

Why?

Very successful in NLP, esp. for large data! Maybe same for vision?

Problem: Image = 2D input, transformer expects 1D sequence

-> need tokenization for images

Naive Approach: flatten the image, each pixel is treated as sequence element

BUT: self-attention has quadratic complexity in sequence length -> not feasible

Transformers for Vision

Why?

Very successful in NLP, esp. for large data! Maybe same for vision?

Problem: Image = 2D input, transformer expects 1D sequence

-> need tokenization for images

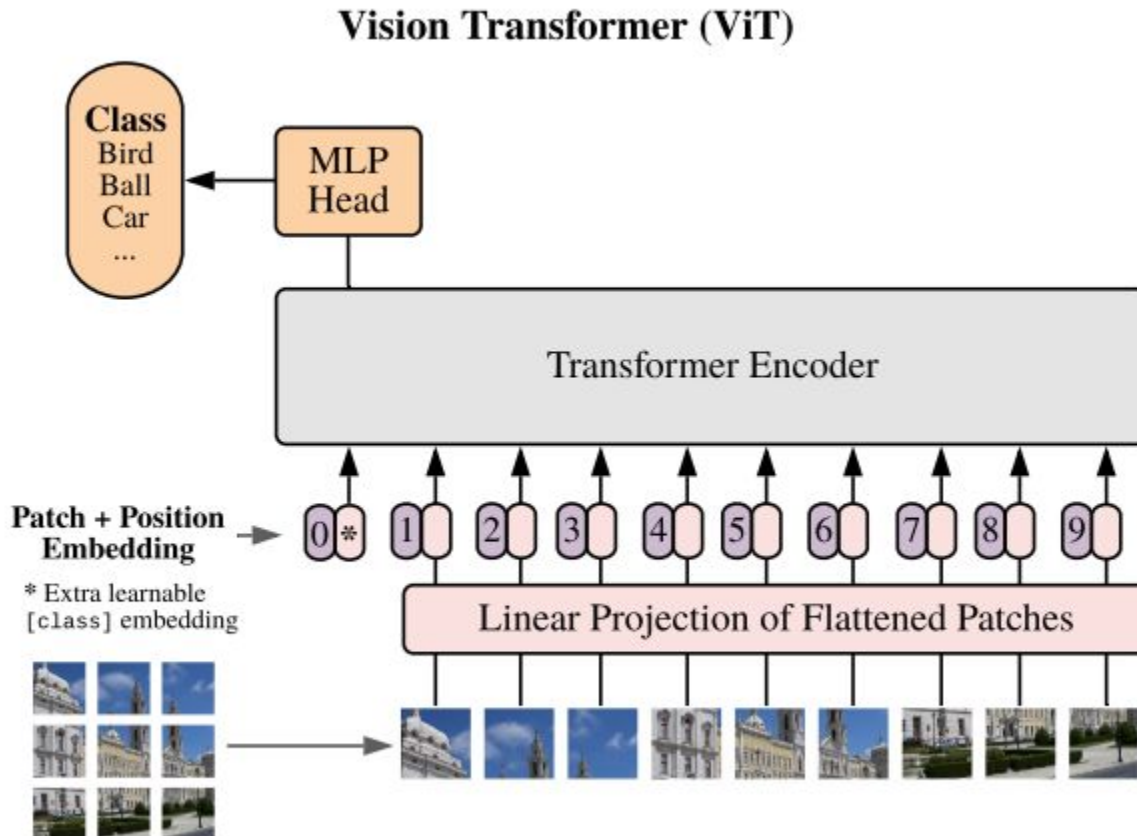
Naive Approach: flatten the image, each pixel is treated as sequence element

BUT: self-attention has quadratic complexity in sequence length -> not feasible

Possible solutions:

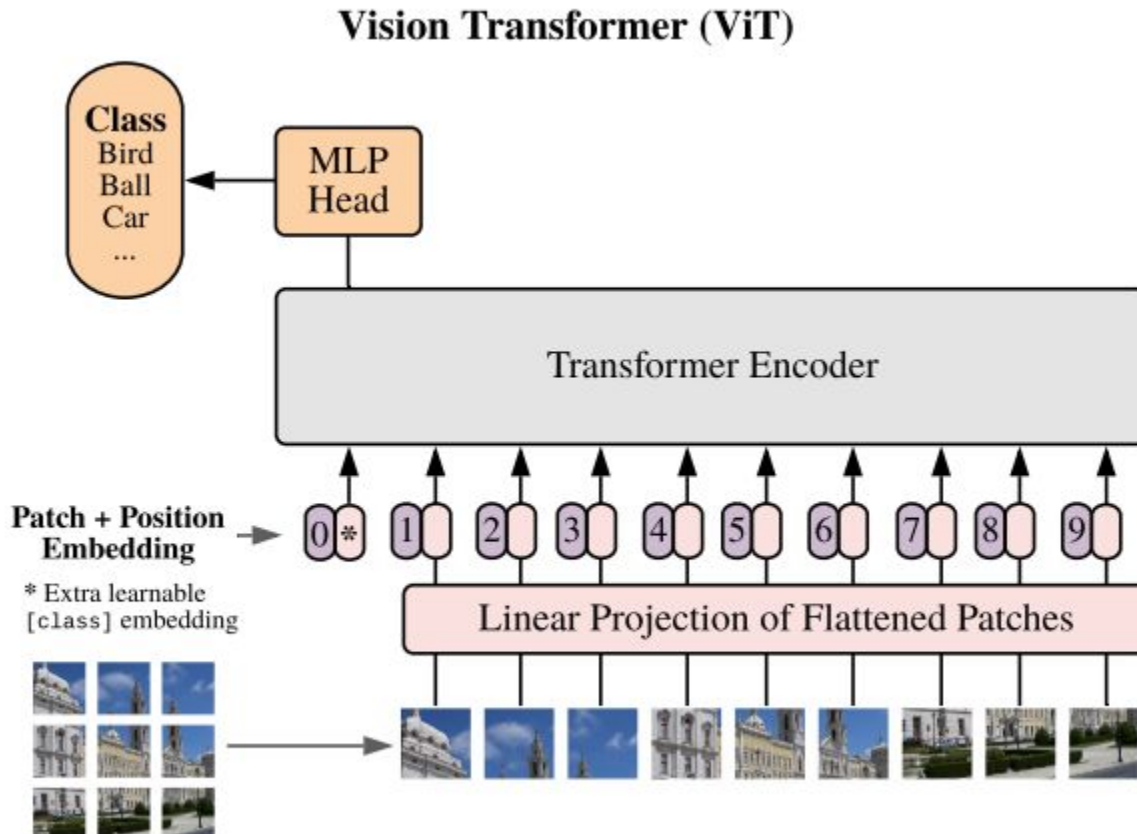
- Apply convolutional back-bone first, use lower resolution feature map
- Use image patches

Vision Transformer (ViT)



“Tokenization” into patches

Vision Transformer (ViT)

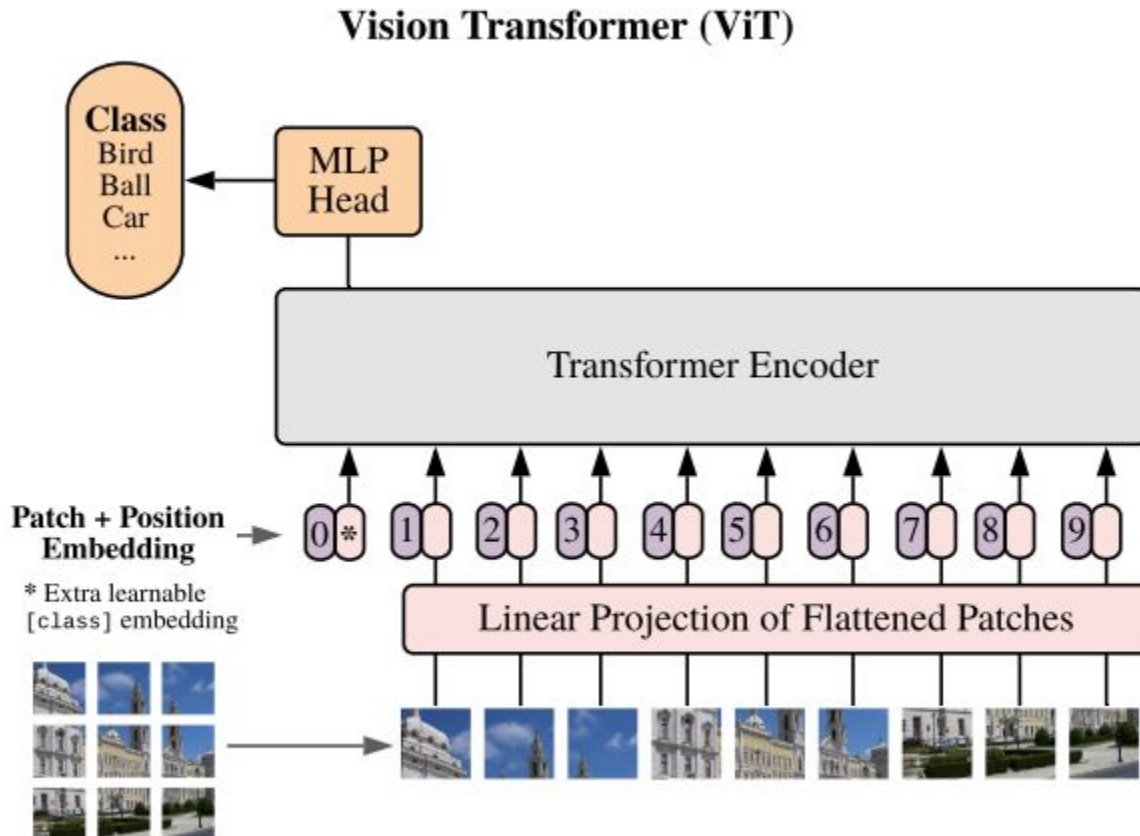


Position embedding

Patch embedding: strided conv.

“Tokenization” into patches

Vision Transformer (ViT)



Classifier head

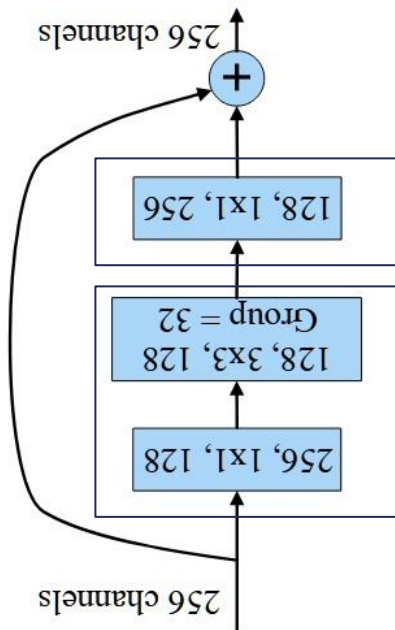
Same as in prev. section

Position embedding

Patch embedding: strided conv.

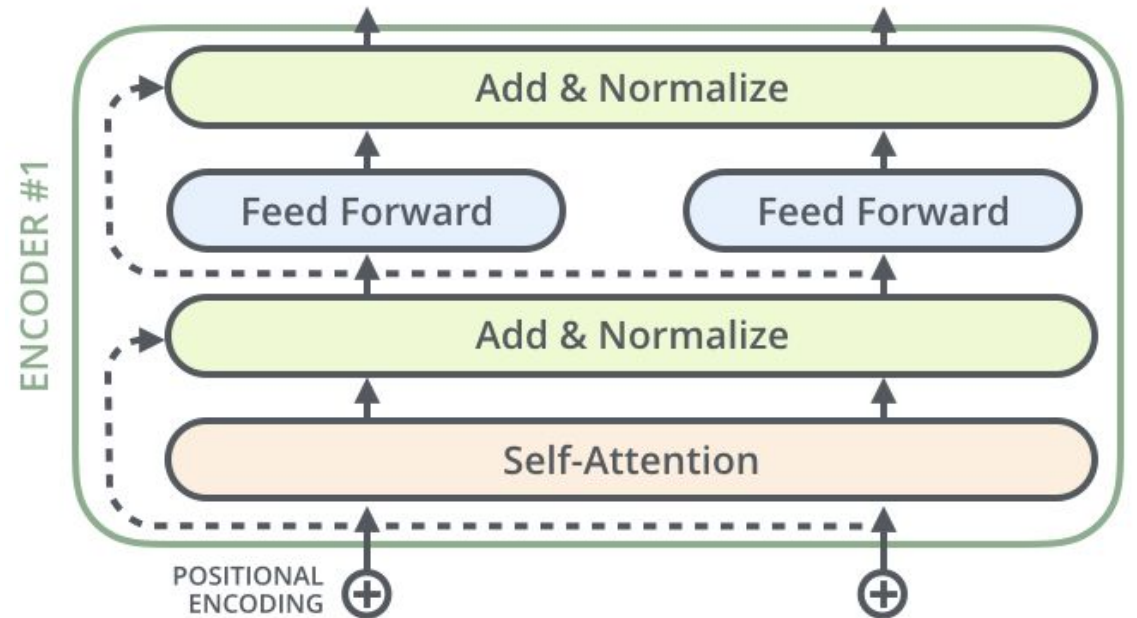
“Tokenization” into patches

CNN vs ViT: Building blocks

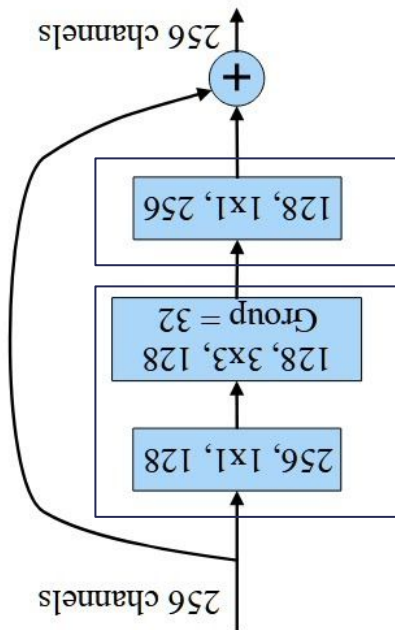


1x1 / FeedForward:

Conv Block (3x3) / Self-Attention:

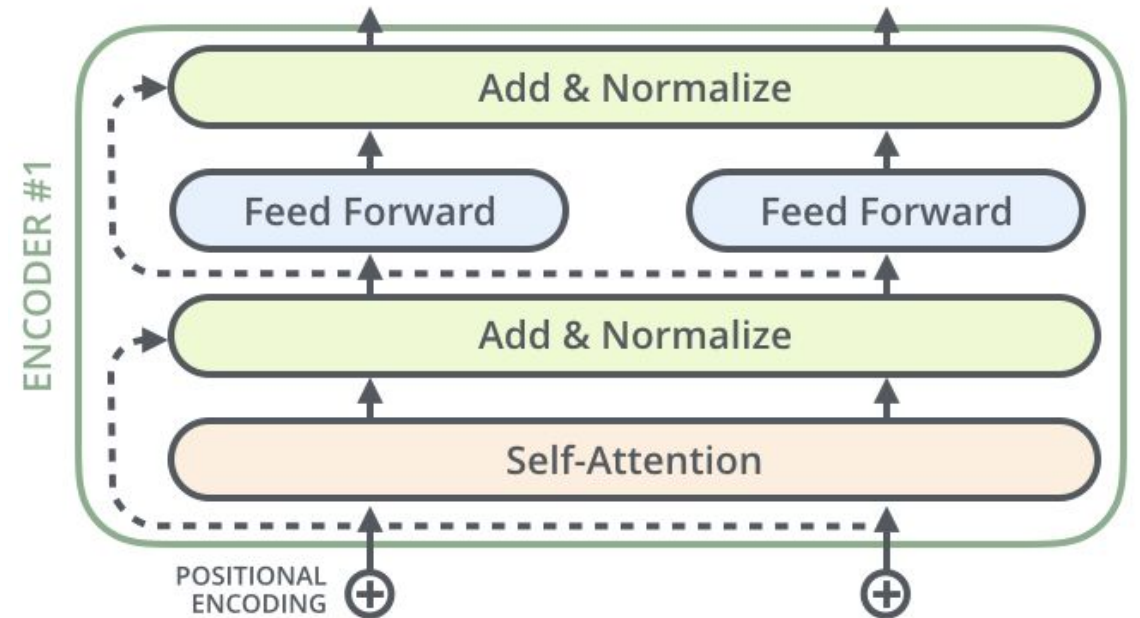


CNN vs ViT: Building blocks



1x1 / FeedForward:
process elements individually
same in CNN / Transformer

Conv Block (3x3) / Self-Attention:
CNN: local field of view
Transformer: global field of view



CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No

CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No
Permutation Invariance	No	Mostly (patch level)

CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No
Permutation Invariance	No	Mostly (patch level)
Spatially local processing	Yes (3x3, 5x5 conv etc.)	Partially (only patch level)

CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No
Permutation Invariance	No	Mostly (patch level)
Spatially local processing	Yes (3x3, 5x5 conv etc.)	Partially (only patch level)
Parameter sharing	Yes	Yes

CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No
Permutation Invariance	No	Mostly (patch level)
Spatially local processing	Yes (3x3, 5x5 conv etc.)	Partially (only patch level)
Parameter sharing	Yes	Yes
Increased depth: lower resolution, wider features	Yes (pyramidal design)	No

CNN vs ViT: Inductive biases

Property	CNN	ViT
Shift invariance	Mostly	No
Permutation Invariance	No	Mostly (patch level)
Spatially local processing	Yes (3x3, 5x5 conv etc.)	Partially (only patch level)
Parameter sharing	Yes	Yes
Increased depth: lower resolution, wider features	Yes (pyramidal design)	No

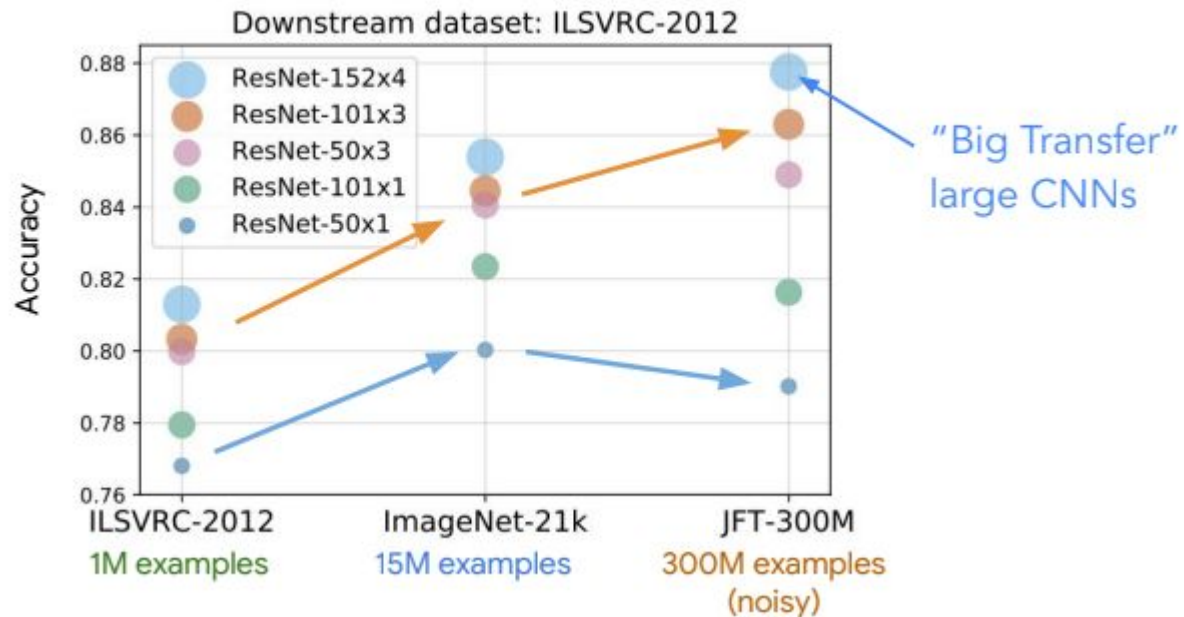
ViT: lacks inductive biases for image data.

- Can it learn to overcome this (with enough training data)?
- If so, does it bring anything? E.g. better performance?

“Scaling Laws”

Larger models: better performance with more training data

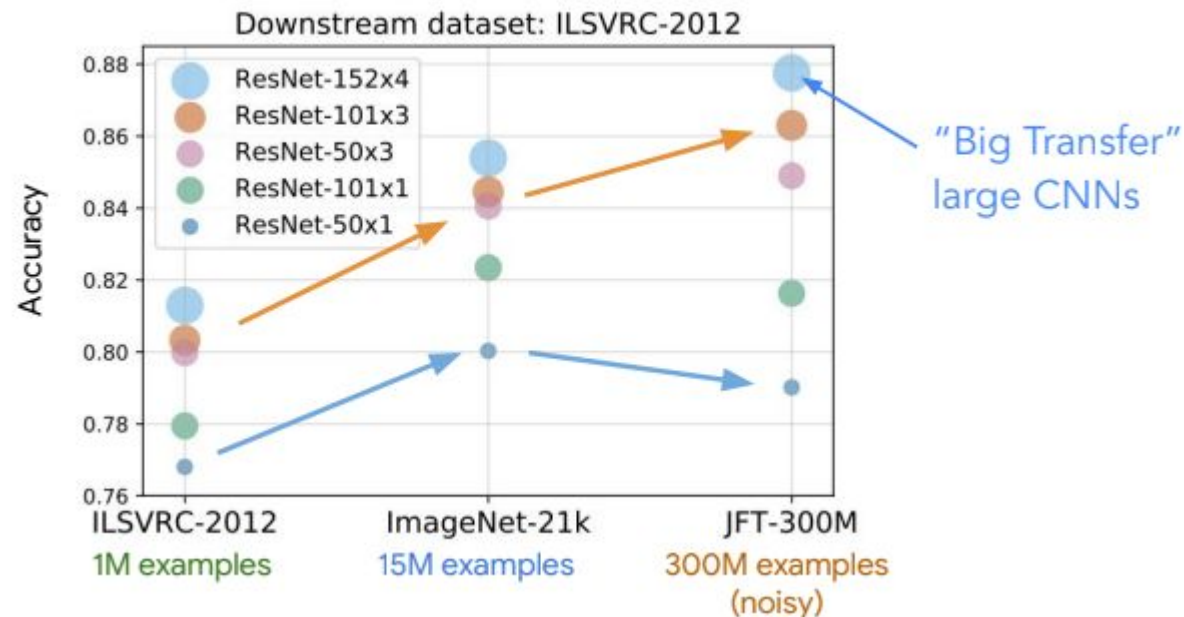
Vision



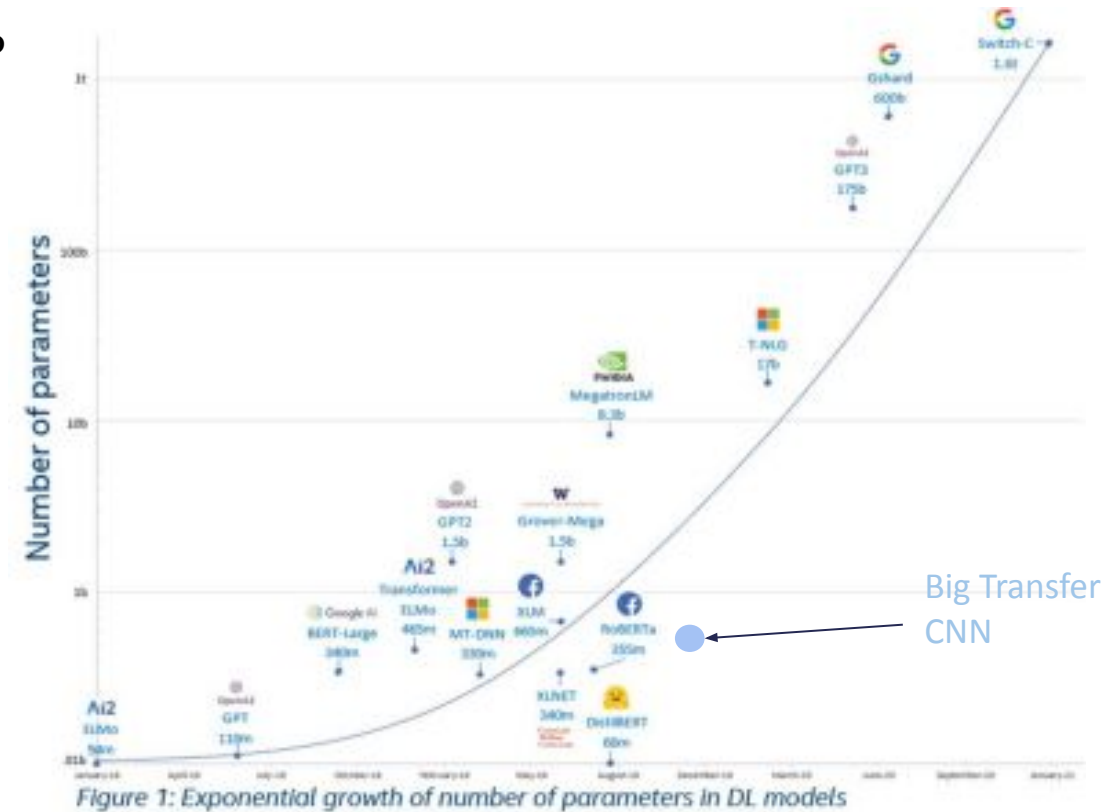
“Scaling Laws”

Larger models: better performance with more training data

Vision



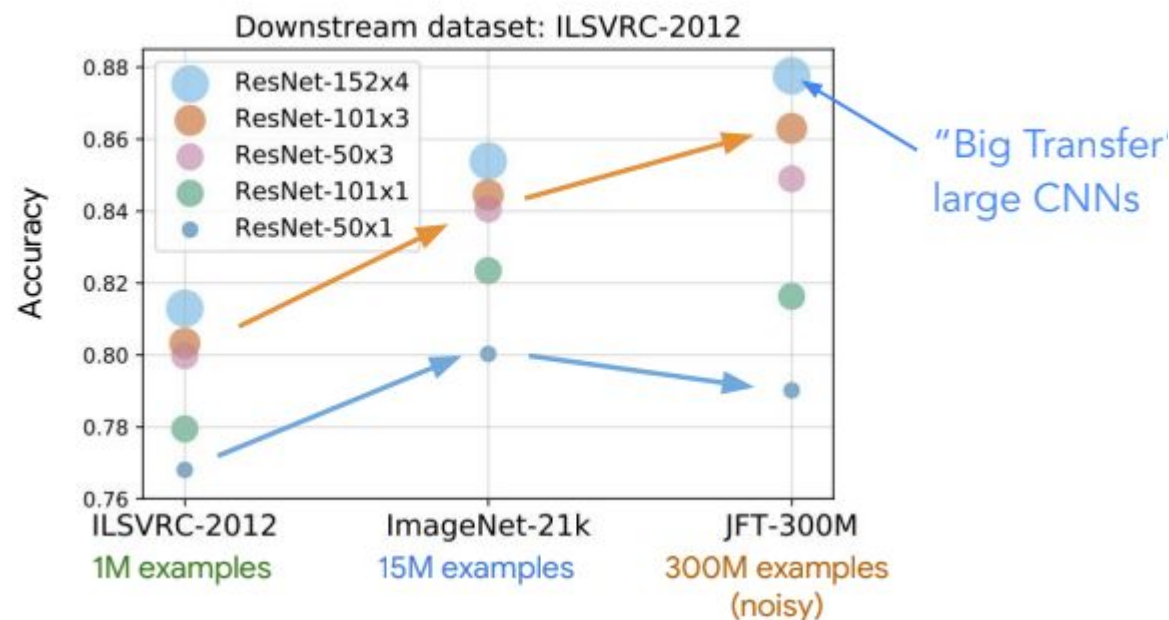
NLP



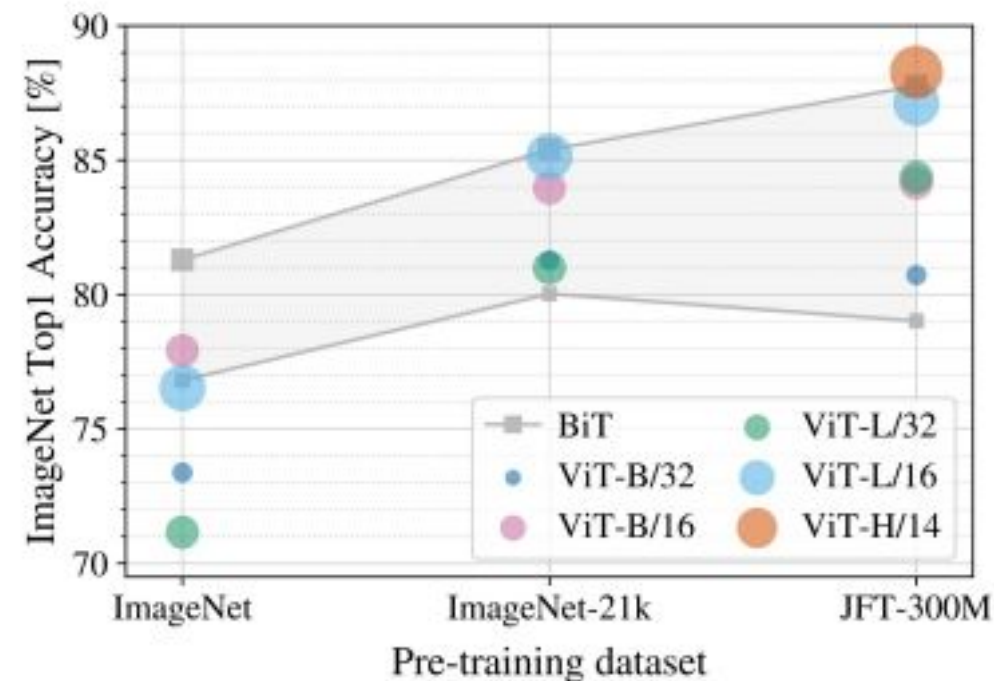
Scaling ViT

BiT = BigTransfer:
Large ResNet

Scaling CNN



Scaling ViT

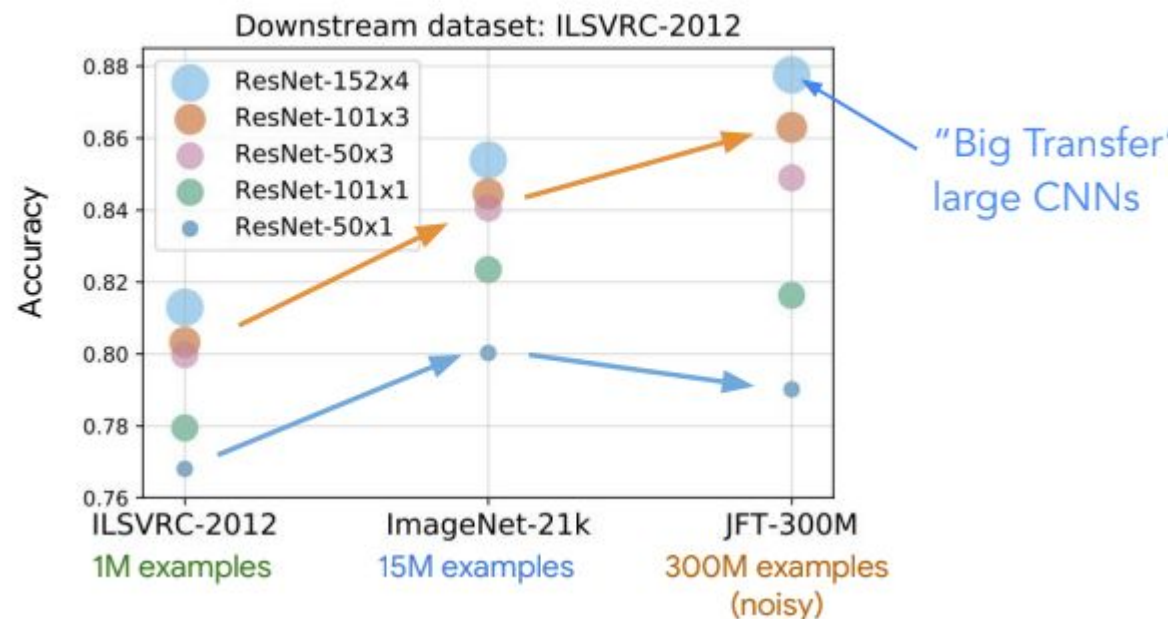


Scaling ViT

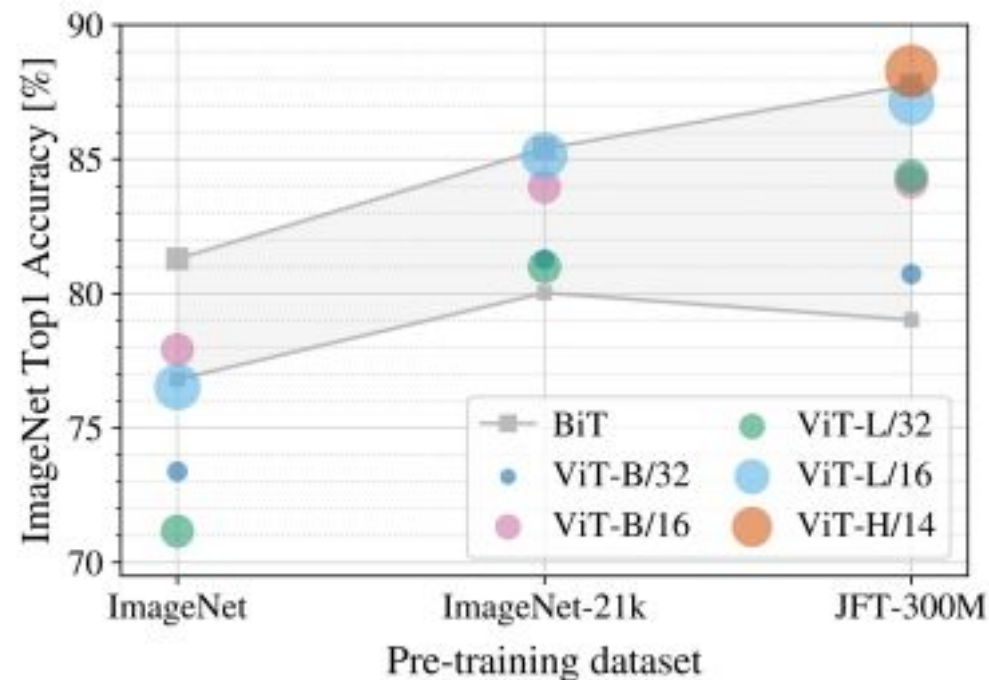
ViT: overfits on ImageNet
but is better with larger pre-training

BiT = BigTransfer:
Large ResNet

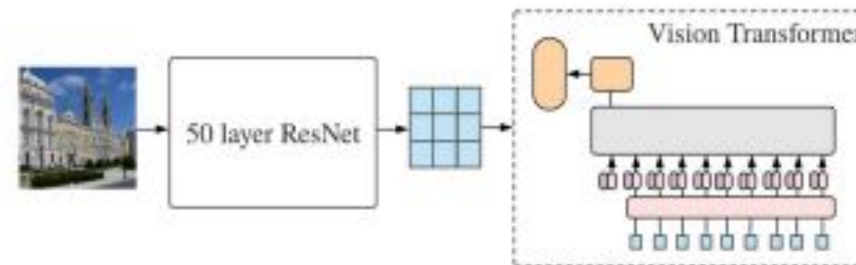
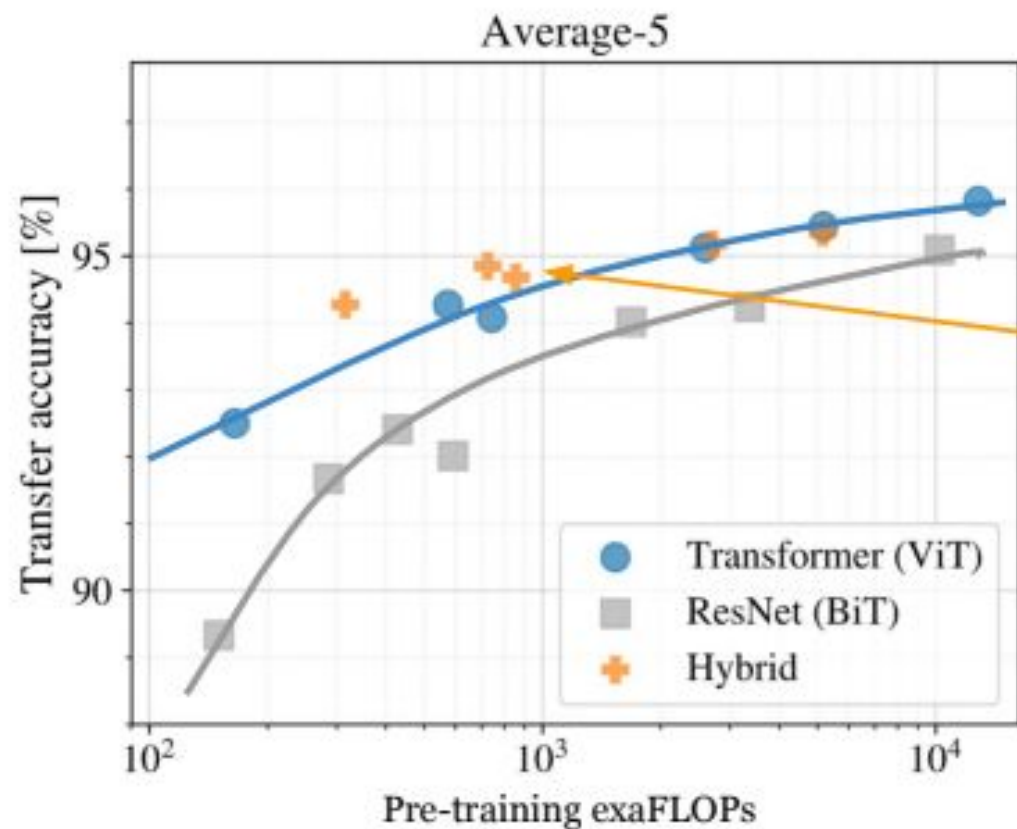
Scaling CNN



Scaling ViT



ViT costs less to pre-train



Hybrid effective at smaller scales

What does attention learn?

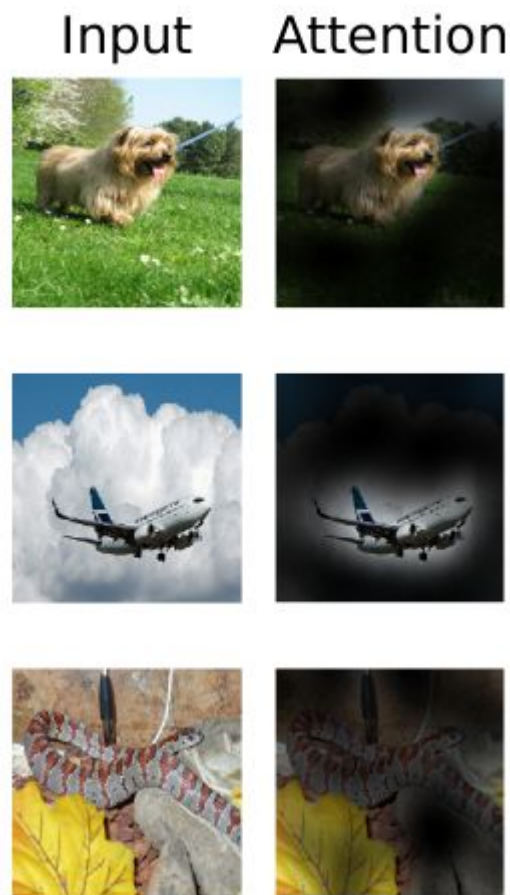
Input



Attention

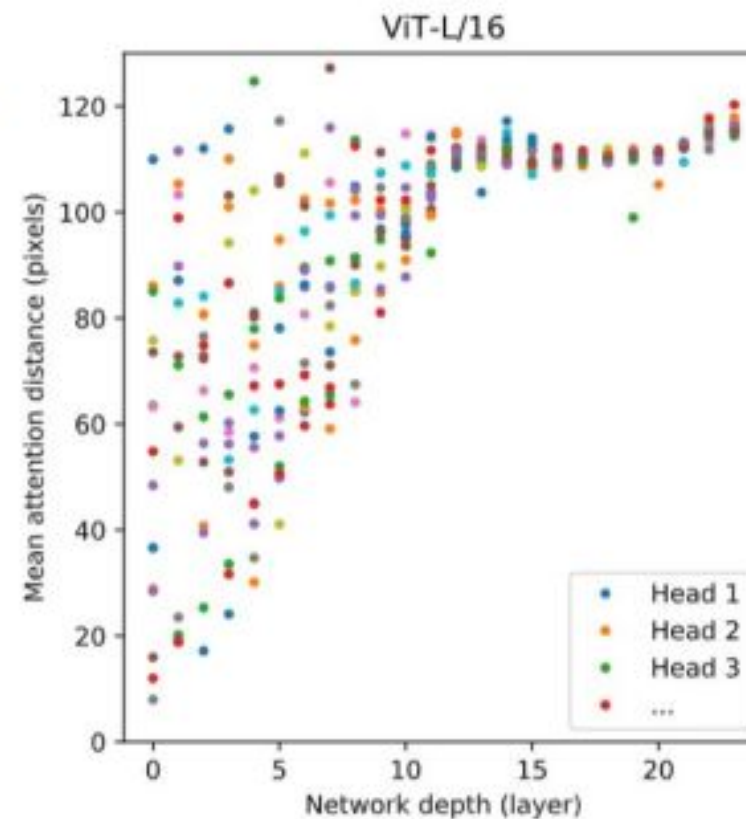


What does attention learn?



Spatially global features

Spatially local features



Inspection of learned attention heads

Vision foundation models: “Large language models for images”

Large language models



Vision foundation models



Segment Anything

CLIP: Connecting
text and images

An astronaut riding
a horse in photorealistic
style.



Dataset



ImageNet
ImageNet Rendition



ObjectNet



ImageNet Sketch

ImageNet ResNet101 CLIP ViT-L

76.2%

76.2%

32.6%

72.3%

25.2%

60.2%

Next lectures

- Lecture 10: Learning without labels: Self-supervised learning
 - And other techniques for (labeled) data efficiency
- Lecture 11: State-of-the-art models for computer vision
 - Vision transformer applications & vision foundation models
- Lecture 12: Architectures for videos & point clouds
- Lecture 13: Diffusion models, flow matching & scene representation

More resources on (vision) transformers

- The illustrated transformer (main source for slides explaining transformers)
 - <https://jalammar.github.io/illustrated-transformer/>
- Lecture and slides by Lucas Beyer (one of the leading researchers on vision transformers)
 - <https://www.youtube.com/watch?v=bMXqnLiVgLk>
 - https://docs.google.com/presentation/d/1ZXFlhYczos679r70Yu8vV9uO6B1J0ztzeDxbnBxD1S0/edit#slide=id.g31364026ad_3_2
- Visualizing transformers and attention
 - <https://poloclub.github.io/transformer-explainer/>

Questions!
